claude-windows / c1fcb4aa-537d-41fd-858e-53f93349bf5a

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\c1fcb4aa-537d-41fd-858e-53f93349bf5a\subagents\workflows\wf_41c8dc8a-8a4\agent-a1189f99b85c3ad56.jsonl`
- SHA-256: `1641995456768802b34c5e0a6243131a11192bf5d228d00f00fa67314517f085`
- Source modified: `2026-07-02T02:53:37+00:00`
- Imported at: `2026-07-05T16:48:24+00:00`
- project: `wf_41c8dc8a-8a4`
- session_id: `c1fcb4aa-537d-41fd-858e-53f93349bf5a`

Transcript

1. user (2026-07-02T02:47:11.980Z)

```
CONTEXT (read carefully):
- You are one auditor in a multi-agent tech-debt/critical-fix audit of the khelsutra-guru multi-repo workspace at C:/Users/avidu/Projects/khelsutra-guru. Today is 2026-07-02. All git repos were just pulled to current origin master/main.
- Repos: badminton-highlight-indexer (the ENGINE, Python, ~1500 tests, org Khelsutra), khelsutra (SPA web/ + marketing site/, TS/React, org avidullu), sports-data-collector, rally-annotator, sports-obsreport (small Python repos). Plus two NON-git dirs: "Annotation Setup", khelsutra-screencast.
- HARD RULES: READ-ONLY. Do NOT modify/create/delete any file, do NOT create branches/worktrees, do NOT pip install, do NOT run the full test suite (CI is trusted on master). You MAY run read-only git and gh commands (gh is authenticated), ruff/mypy IF already installed, and fast read-only scripts.
- EVERY claim must cite file:line (or a gh URL). A claim without file:line evidence is a hypothesis - mark it as such or drop it. The codebase drifts between sessions: re-verify anything you believe from docs against actual code.
- OWNER-GATED areas (flag as owner_gated=true, still report): alpha/serving go-live + Cloudflare dashboard work, Studio P10/P11 (corpus promotion + train/reeval), product/strategy/licensing decisions, real GPU/cloud spend, counsel ToS/DPA, repo rename off "badminton".
- Known-open items from project memory (VERIFY current state, don't trust blindly): engine PR #390 (D1 split main.py, open since 06-26) · audit doc docs/CODE_CLEANUP_AUDIT_2026-06-24.md remaining items D1, D13 (ByteTrack->trackers), Phase-3 (B3/B4/B6/B7/B8/D5/D6/D8-D10/O3/O4) · docs/DECISION_SNAPSHOT_REGRESSION.md next=P1 · docs/GOLDEN_REGRESSION_FIXTURES.md backlog M4/P0-rename/O1/O2 · engine issues #340 (Gemini re-bill), #332 (NSFW preflight), #349 (CPU-bound decode), #384 (CI single-source) · khelsutra issues #97-#114 UX/alpha backlog · DO droplet RETIRED 2026-06-25 (docs may still reference it; api.khelsutra.guru CNAME/CF-Access cleanup was pending).
- Categorize each finding: critical-fix (latent bug/correctness/data-loss/cost-leak), security, tech-debt (structure/duplication/dead code), test-gap, ci-infra, doc-rot, hygiene.
- Severity: P0 = latent bug or cost/data/security risk reachable in normal use; P1 = high-value debt blocking velocity or a stale-open thread; P2 = worthwhile cleanup; P3 = nice-to-have (report max 3 of these).
- Be selective: max ~12 findings, each genuinely worth acting on. Note tracked_in when an issue/PR/doc already tracks it (tracked is NOT done - still report). Your final message is data for an orchestrator, not prose for a human.

YOUR DIMENSION: security + configuration debt in the ENGINE. Repo:
C:/Users/avidu/Projects/khelsutra-guru/badminton-highlight-indexer.
Check: hardcoded bucket/project/region literals outside the single-source config (deploy/gcp/buckets.env landed in #410 - grep for khelsutra-rally-corpus, gen-lang-client, hardcoded regions/zones in backend/ deploy/ scripts/ tools/); secret handling (grep for GEMINI_API_KEY, tokens, any literal secrets - also in docs and artifacts); the path-jail + exposure posture (backend/config/startup.py - any gap vs issue #327's scheme-exempt ask); CORS/auth defaults (auth.py, RALLY_CORS_ALLOW_ORIGINS); pip dependency pins + license posture
```

(pyproject - anything AGPL/unclear per ADR-005); .gitignore correctness (the UTF-16 corruption was fixed in #377 - verify artifacts/ and scratch/ are properly ignored AND untracked).

2. user (2026-07-02T02:47:22.784Z)

```
true
e410136 api: /api/videos surfaces fps + resolution (from the stored ffprobe details) (#413)
81a78bf api: /api/videos returns video_id + a friendly match_name (fix disappearing name) (#412)
8b16cf4 docs: reflect today's storage endpoints + the corpus/user-video boundary (#411)
```

3. user (2026-07-02T02:47:23.703Z)

```
deploy\gcp\ops-agent-startup.sh
deploy\gcp\buckets.env
deploy\gcp\create_gpu_vm.sh
deploy\gcp\README.md
deploy\gcp\provision.sh
deploy\gcp\teardown.sh
```

4. user (2026-07-02T02:47:30.912Z)

```
# shellcheck shell=bash
# Single source of truth for the GCP deploy layer's PROJECT + BUCKET names.
# Every deploy/gcp script `source`s this, so a rename ? or standing up a NEW deployment / region ? is
# ONE edit here, not a hunt across scripts. Override any value via the environment before invoking a
# script (these are just the defaults; a caller-set env value always wins).
#
# DATA-FAMILY SEPARATION (never conflate ? this is a compliance boundary, not tidiness):
#   * TRAINING CORPUS   ? golden clips/labels + WASB weights, PERMANENT (no TTL)   -> RALLY_CORPUS_BUCKET
#   * NIGHTLY/dev OUTPUT ? reports, repo tarball, Cloud Build staging, 30-day TTL  -> RALLY_NIGHTLY_BUCKET
#   * USER-UPLOADED videos are a SEPARATE surface entirely ? the engine's StorageConfig
#     (backend/config/models.py) and the per-request `medium_uri` of POST /api/assets ? and must NEVER
#     land in these buckets. Keep them physically apart so consent / no-minors / not-training holds at
#     the storage layer, not on a promise.
#
# PER-DEPLOYMENT / PER-REGION: a regional deploy just sets RALLY_GCP_PROJECT + RALLY_CORPUS_BUCKET
# (+ RALLY_NIGHTLY_BUCKET) to its own values (e.g. an EU box -> gs://khelsutra-corpus-eu). No script
# or code change ? that is the whole point of centralizing the names here.

# GCP project every bucket + the worker service account live under.
export RALLY_GCP_PROJECT="${RALLY_GCP_PROJECT:-gen-lang-client-0306026826}"

# Training corpus (permanent, no TTL). Canonical name = RALLY_CORPUS_BUCKET. The two legacy names that
# used to be typed independently ? RALLY_GCS_BUCKET (provision.sh) and RALLY_BUCKET (launch.sh) ? are
# still honored for back-compat, then re-exported to the canonical value so any old invocation OR a
# script still reading either name resolves to the SAME bucket (the divergence that caused drift).
export RALLY_CORPUS_BUCKET="${RALLY_CORPUS_BUCKET:-${RALLY_GCS_BUCKET:-${RALLY_BUCKET:-khelsutra-rally-corpus}}}"
export RALLY_GCS_BUCKET="$RALLY_CORPUS_BUCKET"   # back-compat alias (was provision.sh's variable)
export RALLY_BUCKET="$RALLY_CORPUS_BUCKET"       # back-compat alias (was launch.sh's
```

```
variable)

# Nightly / dev EPHEMERAL outputs (30-day TTL; isolated from the corpus so a bucket-wide
TTL can never
# touch golden data ? that blast-radius isolation is deliberate).
export RALLY_NIGHTLY_BUCKET="${RALLY_NIGHTLY_BUCKET:-khelsutra-rally-nightly}"

# WASB weights object. Lives UNDER the corpus bucket, so it is derived from
RALLY_CORPUS_BUCKET (a
# corpus rename carries the weights path with it); override RALLY_WEIGHTS_URI to point
elsewhere.
export RALLY_WEIGHTS_URI="${RALLY_WEIGHTS_URI:-
gs://${RALLY_CORPUS_BUCKET}/models/wasb_badminton_best.pth.tar}"

```

5. user (2026-07-02T02:47:34.400Z)

```
deploy\gcp\buckets.env:20:export RALLY_GCP_PROJECT="${RALLY_GCP_PROJECT:-gen-lang-
client-0306026826}"
deploy\gcp\buckets.env:26:export
RALLY_CORPUS_BUCKET="${RALLY_CORPUS_BUCKET:-${RALLY_GCS_BUCKET:-${RALLY_BUCKET:-khelsutra-rally-
corpus}}}"
deploy\gcp\nightly_regression\create_nightly_bucket.sh:5:#   gs://khelsutra-rally-corpus   ?
PERMANENT, NO TTL: the golden clips + labels + WASB weights
eval_baselines\reelcount_manifest.json:15:      "uri": "gs://khelsutra-rally-corpus/nightly-
inputs/videos/GX030094.mp4",
eval_baselines\reelcount_manifest.json:16:      "labels_uri": "gs://khelsutra-rally-
corpus/nightly-inputs/labels/GX030094_golden_features.json"
eval_baselines\reelcount_manifest.json:20:      "uri": "gs://khelsutra-rally-corpus/nightly-
inputs/videos/Badminton_BXH_2.mp4",
eval_baselines\reelcount_manifest.json:21:      "labels_uri": "gs://khelsutra-rally-
corpus/nightly-inputs/labels/Badminton_BXH_2_golden_features.json"
eval_baselines\reelcount_manifest.json:25:      "uri": "gs://khelsutra-rally-corpus/nightly-
inputs/videos/GX010128.mp4",
eval_baselines\reelcount_manifest.json:26:      "labels_uri": "gs://khelsutra-rally-
corpus/nightly-inputs/labels/GX010128_golden_features.json"
deploy\gcp\README.md:33:| Project | `gen-lang-client-0306026826` (billing **Khelsutra Billing**
`01420D-419EE0-53D83C`) | ? | n/a (shared Gemini project) |
deploy\gcp\README.md:34:| GCS bucket | `gs://khelsutra-rally-corpus` (asia-south1, UBLA) |
~$0.02/GB/mo storage | `gcloud storage rm -r` (? data loss) |
deploy\gcp\README.md:35:| Service account | `rally-worker@gen-lang-
client-0306026826.iam.gserviceaccount.com` (`roles/storage.objectAdmin`) | $0 | `gcloud iam
service-accounts delete` |
deploy\gcp\README.md:51:1. **Service Usage API** ?
https://console.developers.google.com/apis/api/serviceusage.googleapis.com/overview?project=gen-
lang-client-0306026826
deploy\gcp\README.md:52:2. **Link a billing account** (a payment method) ?
https://console.cloud.google.com/billing/linkedaccount?project=gen-lang-client-0306026826
deploy\gcp\README.md:73:gcloud compute project-info describe --project gen-lang-
client-0306026826 \
deploy\gcp\README.md:105:  --project=gen-lang-client-0306026826 \
deploy\gcp\README.md:113:  --service-account=rally-worker@gen-lang-
client-0306026826.iam.gserviceaccount.com \
deploy\gcp\README.md:132:P=gen-lang-client-0306026826; SA=rally-
worker@$P.iam.gserviceaccount.com
deploy\gcp\README.md:206:  "storage": { "backend": "local", "gcs": { "project": "gen-lang-
client-0306026826" } } }
deploy\gcp\README.md:211:#   videos -> gs://khelsutra-rally-corpus/videos/   labels ->
gs://khelsutra-rally-corpus/labels/
deploy\gcp\README.md:216:  --corpus-uri gcs://khelsutra-rally-corpus --out-uri gcs://khelsutra-
rally-corpus \
deploy\gcp\README.md:221:  --video-uri gcs://khelsutra-rally-corpus/videos/GX010135_proxy.mp4 \
deploy\gcp\README.md:222:  --out-uri  gcs://khelsutra-rally-corpus --segmenter wasb_hybrid
--config config.json
deploy\gcp\README.md:227:- **GCS bucket (recommended, co-located):** `gcs://khelsutra-rally-
```

```
corpus/videos/clip.mp4`. Upload with
deploy\gcp\README.md:228:  `gcloud storage cp clip.mp4 gs://khelsutra-rally-corpus/videos/`.
deploy\gcp\README.md:264:`gcloud storage rm -r gs://khelsutra-rally-corpus`.
deploy\gcp\nightly_regression\README.md:30:All management commands run from a machine with
`gcloud` authed to `gen-lang-client-0306026826` (e.g. a
deploy\gcp\nightly_regression\README.md:38:shown). `J=nightly-regression-launcher`, `S=nightly-
regression`, `R=asia-south1`, `P=gen-lang-client-0306026826`.
deploy\gcp\nightly_regression\README.md:63:| `gs://khelsutra-rally-corpus` | **none
(permanent)** | golden **clips + labels** (`nightly-inputs/`) + WASB **weights** (`models/`) |
precious read-only INPUTS ? never auto-deleted |
deploy\gcp\nightly_regression\README.md:87:gcloud storage cp <clip>.mp4 gs://khelsutra-rally-
corpus/nightly-inputs/videos/           # + labels to .../labels/
deploy\gcp\nightly_regression\README.md:98:gcloud run jobs execute nightly-regression-launcher
--region asia-south1 --project gen-lang-client-0306026826
deploy\gcp\nightly_regression\README.md:124:gcloud artifacts docker images delete asia-
south1-docker.pkg.dev/gen-lang-client-0306026826/rally/nightly-launcher --delete-tags
scripts\verify_compute_path.py:15:      --video gs://khelsutra-rally-
corpus/videos/mahadevpura_smoke_180s.mp4 \
scripts\verify_compute_path.py:17:      --region asia-southeast1 --project gen-lang-
client-0306026826
tests\test_workspace.py:107:        cloud = for_video("gs://khelsutra-rally-corpus", VID,
prefix="workspaces")
tests\test_workspace.py:110:            rel_cloud = cloud.path(sub)[len("gs://khelsutra-rally-
corpus") :]
```

6. user (2026-07-02T02:47:45.071Z)

```
deploy\gcp\teardown.sh:14:# Cross-region VM: set the zone, e.g.  RALLY_GCP_ZONE=asia-
southeast1-a bash deploy/gcp/teardown.sh
deploy\gcp\teardown.sh:26:ZONE="${RALLY_GCP_ZONE:-asia-south1-a}"
deploy\gcp\teardown.sh:36:    echo "== stopping $VM ? ? the DISK STILL BILLS (~\$/mo); use
'delete' for \$0. Resume: gcloud compute instances start $VM --zone=$ZONE =="
deploy\gcp\teardown.sh:37:    gcloud compute instances stop "$VM" --zone="$ZONE" --project
"$PROJECT" || echo "  (no running VM named $VM)"
deploy\gcp\teardown.sh:41:    gcloud compute instances delete "$VM" --zone="$ZONE" --project
"$PROJECT" --quiet || echo "  (no VM named $VM)"
deploy\gcp\teardown.sh:46:    gcloud compute instances delete "$VM" --zone="$ZONE" --project
"$PROJECT" --quiet || echo "  (no VM)"
deploy\cloudrun\cloudbuild.yaml:11:  _REGION: asia-southeast1   # L4 GPU is region-limited
(asia-southeast1 / europe-west1 / europe-west4 / us-central1 / us-east4)
deploy\gcp\provision.sh:2:# Provision the DECOUPLED_COMPUTE GCP storage + IAM (asia-south1 /
Mumbai). Idempotent: safe to
deploy\gcp\provision.sh:15:REGION="${RALLY_GCP_REGION:-asia-south1}"
deploy\gcp\provision.sh:33:echo "== GCS bucket (asia-south1, uniform bucket-level access) =="
deploy\gcp\create_gpu_vm.sh:7:# It sweeps zones for GPU capacity ? asia-south1 (Mumbai, co-
located with the bucket) first, then
deploy\gcp\create_gpu_vm.sh:8:# asia-southeast1 (Singapore), which L4 frequently falls back to
when Mumbai is stocked-out.
deploy\gcp\create_gpu_vm.sh:31:ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-
south1-c asia-southeast1-a asia-southeast1-b asia-southeast1-c}"
deploy\gcp\create_gpu_vm.sh:42:  if gcloud compute instances create "$NAME" --project="$PROJECT"
--zone="$Z" \
deploy\gcp\create_gpu_vm.sh:51:    echo "   SSH:    gcloud compute ssh $NAME --zone=$Z"
deploy\gcp\nightly_regression\create_nightly_bucket.sh:21:LOCATION="${RALLY_GCP_LOCATION:-asia-
south1}"
deploy\gcp\nightly_regression\register_scheduler.sh:21:REGION="${RALLY_RUN_REGION:-asia-south1}"
deploy\gcp\nightly_regression\register_scheduler.sh:56:gcloud run jobs deploy "$JOB"
--project="$PROJECT" --region="$REGION" \
deploy\gcp\nightly_regression\register_scheduler.sh:73:       gcloud run jobs execute $JOB
--region=$REGION --project=$PROJECT
deploy\gcp\nightly_regression\register_scheduler.sh:74:   Watch it:   gcloud run jobs executions
list --job=$JOB --region=$REGION --project=$PROJECT
deploy\gcp\nightly_regression\register_scheduler.sh:75:               gcloud beta run jobs
executions logs read <execution> --region=$REGION   # launcher logs
deploy\gcp\nightly_regression\register_scheduler.sh:76:               gcloud compute instances
```

```
get-serial-port-output <vm> --zone=<zone>        # the GPU run
deploy\gcp\nightly_regression\register_scheduler.sh:78:   Remove:     gcloud scheduler jobs
delete $SCHED --location=$REGION; gcloud run jobs delete $JOB --region=$REGION
deploy\gcp\nightly_regression\register_scheduler.sh:83:  gcloud run jobs execute "$JOB"
--region="$REGION" --project="$PROJECT"
deploy\gcp\nightly_regression\launch.sh:14:# ? The VM-side flow is NOT covered by CI. Tail:
gcloud compute instances get-serial-port-output <name> --zone=<z>.
deploy\gcp\nightly_regression\launch.sh:40:ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b
asia-south1-c asia-southeast1-a asia-southeast1-b asia-southeast1-c}"
deploy\gcp\nightly_regression\launch.sh:68:  if gcloud compute instances create "$NAME"
--project="$PROJECT" --zone="$Z" \
deploy\gcp\nightly_regression\launch.sh:78:    echo "   serial log: gcloud compute instances
get-serial-port-output $NAME --zone=$Z --project=$PROJECT"
deploy\gcp\nightly_regression\launch.sh:79:    echo "   if it doesn't self-delete: gcloud
compute instances delete $NAME --zone=$Z -q"
deploy\gcp\nightly_regression\run_on_vm.sh:39:    echo "   ? MANUAL CLEANUP REQUIRED: gcloud
compute instances list  then  ... delete <name> --zone=<zone> -q"
deploy\gcp\nightly_regression\run_on_vm.sh:43:  gcloud compute instances delete "$NAME"
--zone="$ZONE" --quiet || \
deploy\gcp\nightly_regression\run_on_vm.sh:44:    echo "WARN: self-delete failed ? DELETE
MANUALLY: gcloud compute instances delete $NAME --zone=$ZONE -q"
```

7. user (2026-07-02T02:47:46.453Z)

```
backend\compute\cloud_run.py:127:        token: Optional[str] = None,
backend\compute\cloud_run.py:136:        # The dispatch token keys the done-marker so concurrent
jobs to one bucket don't collide.
backend\compute\cloud_run.py:138:        self._token = token
backend\compute\cloud_run.py:147:        token = self._token or uuid.uuid4().hex
backend\compute\cloud_run.py:148:        done_uri = f"{out_root}/_dispatch/{token}.done"
backend\compute\__init__.py:40:    token: Optional[str] = None,
backend\compute\__init__.py:77:        token=token,
backend\api\auth.py:8:token's ``email``, else ``sub``).
backend\api\auth.py:33:    """A required Cf-Access token was missing or failed verification ?
the caller returns 401/403."""
backend\api\auth.py:69:def verify_cf_access_token(
backend\api\auth.py:70:    token: str, *, team_domain: str, aud: str, jwks: dict
backend\api\auth.py:86:        kid = jwt.get_unverified_header(token).get("kid")
backend\api\auth.py:88:            raise EdgeAuthError("token header has no kid")
backend\api\auth.py:91:            token,
backend\api\auth.py:105:        raise EdgeAuthError(f"Cf-Access token verification failed:
{exc}") from exc
backend\api\auth.py:109:        raise EdgeAuthError("verified token carries no email/sub claim")
backend\api\auth.py:140:    token = headers.get(CF_ACCESS_HEADER) or
headers.get(CF_ACCESS_HEADER.lower())
backend\api\auth.py:141:    if not token:
backend\api\auth.py:147:    return verify_cf_access_token(token, team_domain=team_domain,
aud=aud, jwks=jwks)
backend\config\models.py:354:    cf_access_aud: str = ""  # the CF Access application AUD tag
the token must carry
backend\config\startup.py:24:no token can verify ? **error**, lifting the current first-request
failure to boot), and the
backend\config\startup.py:121:    # (2) Edge auth cannot verify a token without the team domain
+ AUD. Today that only fails on the
backend\config\startup.py:141:    # (3) The auth extra (PyJWT[crypto]) is lazy-imported by
auth.verify_cf_access_token. If it's
backend\assets\fonts\README.md:17:basic Latin, so the Latin brand token ("KhelSutra.guru")
renders correctly alongside the local script.
backend\main.py:461:        "gemini_key_present": bool(os.environ.get("GEMINI_API_KEY")),
backend\main.py:1817:        api_key = os.environ.get("GEMINI_API_KEY")
backend\main.py:1818:        if not api_key:
backend\main.py:1821:            "gemini", api_key=api_key, model_name=config.ai_model
backend\reporting\spec.yaml:29:      - { path: "stages.ai_handoff.details.api_key_present", op:
eq, value: false }
backend\reporting\spec.yaml:35:      CREDENTIAL, not 'the video has no rallies'. Fix: put
```

```
GEMINI_API_KEY=<key> in a
backend\reporting\spec.yaml:69:      - { path: "stages.ai_handoff.details.api_key_present", op:
eq, value: true }
backend\storage\gdrive_api.py:16:consumer "connect your Drive" flow: client id/secret + refresh
tokens) is OWNER-gated** ? this backend
backend\storage\gdrive_api.py:205:        page_token: Optional[str] = None
backend\storage\gdrive_api.py:212:                    fields="nextPageToken, files(id, name)",
backend\storage\gdrive_api.py:213:                    pageToken=page_token,
backend\storage\gdrive_api.py:220:            page_token = resp.get("nextPageToken")
backend\storage\gdrive_api.py:221:            if not page_token:
backend\reporting\harvest.py:158:    key_env = f"{provider.upper()}_API_KEY"
backend\reporting\harvest.py:167:                "api_key_present":
bool(os.environ.get(key_env)) if ai_based else None,
backend\storage\s3.py:9:/ ``AWS_SESSION_TOKEN`` env, instance role, or ``~/.aws``). A ``client``
may be injected
backend\eval\golden_fixtures.py:121:#: / source-path token), so it is rejected up-front by
construction (red-team: free-text id leak).
backend\eval\gemini_refine.py:17:Run (GEMINI_API_KEY in env):
backend\eval\gemini_refine.py:73:    api_key: str,
backend\eval\gemini_refine.py:86:    provider = provider_registry.get("gemini", api_key=api_key,
model_name=model)
backend\eval\gemini_refine.py:138:    api_key = os.environ.get("GEMINI_API_KEY")
backend\eval\gemini_refine.py:139:    if not api_key:
backend\eval\gemini_refine.py:140:        raise SystemExit("GEMINI_API_KEY not set in env")
backend\eval\gemini_refine.py:161:                api_key,
backend\eval\gemini_refine.py:230:    api_key = os.environ.get("GEMINI_API_KEY")
backend\eval\gemini_refine.py:231:    if not api_key:
backend\eval\gemini_refine.py:232:        raise SystemExit("GEMINI_API_KEY not set in env")
backend\eval\gemini_refine.py:256:                "gemini", api_key=api_key, model_name=model
backend\utils\app_home.py:62:    """``<app-home>/.env`` ? the dotenv file the server prefers
(e.g. ``GEMINI_API_KEY``)."""
backend\providers\__init__.py:16:    def get(self, name: str, api_key: str, model_name: str) ->
AIVideoProvider:
backend\providers\__init__.py:21:        return self._registry[name](api_key=api_key,
model_name=model_name)
backend\ui\assets\index-ukuS9wxa.js:57:[Omitted long matching line]
backend\ui\assets\index-ukuS9wxa.js:60:[Omitted long matching line]
backend\eval\golden_real_fixtures.py:80:#: The fixed schema filenames the writers ALWAYS emit
(as ``paths`` literals). A source-path token
backend\eval\golden_real_fixtures.py:100:    (b) a 32-hex MD5 (the ``video_id`` shape), or (c)
any identifying token of a source path
backend\eval\golden_real_fixtures.py:123:        for token in (sp, p.name, p.stem):
backend\eval\golden_real_fixtures.py:124:            t = (token or "").strip().lower()
backend\eval\golden_real_fixtures.py:125:            # Skip ONLY a token that EXACTLY equals a
schema-literal the writers always emit
backend\eval\golden_real_fixtures.py:127:            # skip on substring containment: a token
like ``label`` or ``traj`` is a SUBSTRING of
backend\eval\golden_real_fixtures.py:134:                    f"{where}: source-path token
{token!r} found in written output ? the source "
backend\eval\golden_real_fixtures.py:199:    contain no FORBIDDEN_KEY / MD5 / source-path token
would slip through), so it never reaches the
backend\providers\gemini.py:38:    def __init__(self, api_key: str, model_name: str):
backend\providers\gemini.py:39:        super().__init__(api_key, model_name)
backend\providers\gemini.py:47:                api_key=self.api_key,
backend\utils\redact.py:10:``edge_auth_enabled`` (a boolean), ``cf_access_aud`` (a public id),
or an ML ``max_tokens`` knob.
backend\utils\redact.py:20:#: the ``*_key`` / ``*_token`` suffixes (verified to hit ZERO current
AppConfig fields ? guarded by
backend\utils\redact.py:22:#: ``edge_auth_enabled``, ``cf_access_aud``, ``max_tokens``, or
``tokenizer_name``.
backend\utils\redact.py:26:
r"|access[_-]?token|auth[_-]?token|refresh[_-]?token|session[_-]?token"
backend\utils\redact.py:27:    r"|_key$|_token$|^key$|^token$)",
backend\providers\base.py:6:    def __init__(self, api_key: str, model_name: str):
backend\providers\base.py:7:        self.api_key = api_key
```

```
backend\pipeline\stitcher\watermark_i18n.py:5:their language. The brand token ("KhelSutra.guru")
stays Latin; only the surrounding phrase localizes.
backend\pipeline\segmenters\gemini.py:102:        api_key_env_var =
f"{provider_name.upper()}_API_KEY"
backend\pipeline\segmenters\gemini.py:103:        api_key = os.environ.get(api_key_env_var)
backend\pipeline\segmenters\gemini.py:104:        if not api_key:
backend\pipeline\segmenters\gemini.py:105:            from backend.pipeline.segmenters._keyhint
import api_key_hint
backend\pipeline\segmenters\gemini.py:108:                f"{api_key_env_var} is not set ? the
{provider_name} segmenter cannot run. "
backend\pipeline\segmenters\gemini.py:109:                f"Set your API key. " +
api_key_hint(api_key_env_var)
backend\pipeline\segmenters\gemini.py:116:                provider_name, api_key=api_key,
model_name=model_name
backend\pipeline\segmenters\trajectory_hybrid.py:203:            api_key_env_var =
f"{provider_name.upper()}_API_KEY"
backend\pipeline\segmenters\trajectory_hybrid.py:204:            api_key =
os.environ.get(api_key_env_var)
backend\pipeline\segmenters\trajectory_hybrid.py:205:            if not api_key:
backend\pipeline\segmenters\trajectory_hybrid.py:206:                from
backend.pipeline.segmenters._keyhint import api_key_hint
backend\pipeline\segmenters\trajectory_hybrid.py:209:                    f"{api_key_env_var}
environment variable is not set! "
backend\pipeline\segmenters\trajectory_hybrid.py:211:                    +
api_key_hint(api_key_env_var)
backend\pipeline\segmenters\trajectory_hybrid.py:216:                    provider_name,
api_key=api_key, model_name=model_name
backend\pipeline\segmenters\yolo_hybrid.py:81:        api_key_env_var =
f"{provider_name.upper()}_API_KEY"
backend\pipeline\segmenters\yolo_hybrid.py:82:        api_key = os.environ.get(api_key_env_var)
backend\pipeline\segmenters\yolo_hybrid.py:83:        if not api_key:
backend\pipeline\segmenters\yolo_hybrid.py:84:            from
backend.pipeline.segmenters._keyhint import api_key_hint
backend\pipeline\segmenters\yolo_hybrid.py:87:                f"{api_key_env_var} environment
variable is not set! "
backend\pipeline\segmenters\yolo_hybrid.py:89:                + api_key_hint(api_key_env_var)
backend\pipeline\segmenters\yolo_hybrid.py:95:                provider_name, api_key=api_key,
model_name=model_name
backend\pipeline\segmenters\_keyhint.py:10:def api_key_hint(var: str) -> str:
backend\pipeline\segmenters\_keyhint.py:13:        return f'In PowerShell:
$env:{var}="your_api_key_here"'
backend\pipeline\segmenters\_keyhint.py:14:    return f"Set it with: export
{var}=your_api_key_here"
```

8. user (2026-07-02T02:47:56.915Z)

```
No matches found
```

9. user (2026-07-02T02:47:57.391Z)

```
1       """Startup checks ? per-profile coherence (M5) + exposure-safety posture (M9 hardening).
2
3       A deployment profile (``truly-local`` / ``cloud-serving`` / ``cpu-mock``) implies a
coherent
4       runtime. These checks **fail/warn fast at process startup** ? a loud signal *before* any
5       (expensive) job runs ? when the active profile is inconsistent with the environment:
6
7         * **cloud-serving with a non-cloud storage backend** ? a hard **error** (the one
config bug we
8           can detect with certainty + zero false-positive risk: a cloud profile that can't
reach a
9           bucket can never serve). This is the README §4.3 "loud error before any job".
10        * a **GPU mismatch** (e.g. ``device='cuda'`` on a box where CUDA isn't visible, or
``device='auto'``
11          falling back to CPU) ? a **warning**, never an error: the probe is best-effort
```

```
(torch may live
    only in the detector subprocess env ? see ``presets.probe_cuda_available``), and the detector's
    own healthcheck is the real authority. A warning surfaces it early without false-failing.
  * **creds that can't be confirmed** (no ``GOOGLE_APPLICATION_CREDENTIALS`` / not on Cloud Run) ?
    a **warning**: Application Default Credentials can come from the metadata server / gcloud, which
    we can't probe offline, so the storage backend's own init is the authority; we just flag it.

**Exposure-safety posture (server only, ``include_exposure=True``).** When the engine is *exposed*
behind an edge gate (the owner flips ``security.edge_auth_enabled`` on for the Cloudflare-Access
boundary ? the alpha-shareable "tunnel-to-a-box" topology), the gate is only half the safety story.
A half-configured exposure starts *looking* healthy then leaks / 500s per request, so we verify the
rest of the posture at **boot**: the ``allowed_video_root`` path-jail is set (else an authenticated
tenant could read an arbitrary local file ? **error**), the CF team-domain + AUD are present (else
no token can verify ? **error**, lifting the current first-request failure to boot), and the
``auth`` extra is importable (else every request 500s ? **warning**). Only the HTTP **server** passes
``include_exposure=True``; the worker reads ``--video-uri`` (not untrusted tenant paths), so it is
never "exposed" and stays unaffected.

**DEFAULT-OFF (both dimensions):** with no profile selected (``profile == ""``) the profile checks are
a strict **no-op** ? exactly the pre-M5 behaviour; with ``edge_auth_enabled=False`` (today's default)
the exposure checks are a strict **no-op** too. So nothing changes for today's manual-knob deployments.
"""


from __future__ import annotations

import logging
import os
from dataclasses import dataclass
from typing import Any, Mapping, Optional

LEVEL_ERROR = "error"
LEVEL_WARN = "warn"

# Storage backends that count as "a cloud bucket" for cloud-serving coherence.
_CLOUD_STORAGE = ("gcs", "s3")
# STRONG env signals that Google ADC *might* resolve. Deliberately NOT GOOGLE_CLOUD_PROJECT /
# CLOUDSDK_CONFIG ? those are often set without usable creds (a plain project id / a config dir
# with no active login), so they would suppress the heads-up falsely. (s3 creds are intentionally
# left to boto3's default chain ? no parallel heads-up, by design.)
_GCP_ENV_HINTS = ("GOOGLE_APPLICATION_CREDENTIALS", "K_SERVICE", "GCE_METADATA_HOST")


@dataclass(frozen=True)
class StartupCheck:
    """One per-profile finding. ``level`` is ``error`` (fatal) or ``warn`` (heads-
```

```
up)."""

    level: str
    code: str  # stable, greppable code (e.g. "STORAGE_INCOHERENT")
    message: str


class StartupCheckError(RuntimeError):
    """Raised by ``enforce_startup_checks`` when an active profile fails a FATAL check."""

    def __init__(self, failures: list[StartupCheck]):
        self.failures = failures
        super().__init__(
            "deployment profile startup check failed: "
            + "; ".join(f"[{c.code}] {c.message}" for c in failures)
        )


def _creds_discoverable(env: Mapping[str, str]) -> bool:
    """True if *some* Google ADC source is hinted by the environment. Best-effort ? a False here
    only downgrades to a WARNING, never a hard failure (the storage init is the authority)."""
    return any(env.get(k) for k in _GCP_ENV_HINTS)


def _auth_extra_available() -> bool:
    """True if PyJWT (the ``auth`` extra) is importable ? probed WITHOUT importing it (find_spec),
    so this stays cheap + side-effect-free. Module-level so tests can monkeypatch it deterministically
    (whether PyJWT is installed in the test env must not change the WARN's presence)."""
    try:
        import importlib.util

        return importlib.util.find_spec("jwt") is not None
    except (
        Exception
    ):  # pragma: no cover - find_spec only raises on a broken parent package
        return False


def _exposure_checks(config: Any, env: Mapping[str, str]) -> list[StartupCheck]:
    """Exposure-safety posture for an *exposed* HTTP server. Gated on ``security.edge_auth_enabled``
    (default False ? ``[]`` = strict no-op, today's behaviour). When edge auth is ON ? the "I am
    exposing this engine behind the CF-Access gate" signal ? the gate alone isn't enough: verify the
    rest of the posture at boot so a half-configured exposure fails fast instead of leaking / 500-ing
    per request. Config-only (no IO beyond the ``find_spec`` auth-extra probe); ``env`` is the
    caller's environment view (so the host-allowlist check is testable)."""
    sec = getattr(config, "security", None)
    if not sec or not getattr(sec, "edge_auth_enabled", False):
        return []  # DEFAULT-OFF: edge auth off ? no posture enforcement (byte-identical to today).

    out: list[StartupCheck] = []

    # (1) The path-jail is load-bearing once authenticated tenants can hit /api/process: WITHOUT it,
    # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset
```

```
= "any local
        # file", the single-user default). Fatal for an exposed engine.
        root = getattr(sec, "allowed_video_root", None)
        if not root or not str(root).strip():
            out.append(
                StartupCheck(
                    LEVEL_ERROR,
                    "EXPOSED_NO_VIDEO_ROOT",
                    "security.edge_auth_enabled is ON (engine exposed) but security.allowed_video_root is "
                    "unset ? an authenticated tenant could make /api/process read an arbitrary local file. "
                    "Set allowed_video_root to a jail directory before exposing the engine.",
                )
            )

        # (2) Edge auth cannot verify a token without the team domain + AUD. Today that only fails on the
        # FIRST request (auth.resolve_tenant); lift it to BOOT so a misconfigured gate never starts
        # "healthy" then 401s every request.
        team = (getattr(sec, "cf_team_domain", "") or "").strip()
        aud = (getattr(sec, "cf_access_aud", "") or "").strip()
        if not team or not aud:
            missing = " + ".join(
                n for n, v in (("cf_team_domain", team), ("cf_access_aud", aud)) if not v
            )
            out.append(
                StartupCheck(
                    LEVEL_ERROR,
                    "EXPOSED_EDGE_AUTH_INCOMPLETE",
                    f"security.edge_auth_enabled is ON but {missing} is not configured ? the Cf-Access JWT "
                    "cannot be verified (every request would fail). Set security.cf_team_domain "
                    "(e.g. https://<team>.cloudflareaccess.com) and security.cf_access_aud (the CF Access "
                    "application AUD tag).",
                )
            )

        # (3) The auth extra (PyJWT[crypto]) is lazy-imported by auth.verify_cf_access_token. If it's
        # missing, every request 500s at verify time ? a boot WARN beats a per-request 500. Warning, not
        # fatal: the auth module's own ImportError is the final authority.
        if not _auth_extra_available():
            out.append(
                StartupCheck(
                    LEVEL_WARN,
                    "EXPOSED_AUTH_EXTRA_MISSING",
                    "security.edge_auth_enabled is ON but PyJWT[crypto] is not importable ? Cf-Access "
                    "verification will fail at request time. Install the 'auth' extra (pip install -e .[auth]).",
                )
            )

        # (4) P0c: an exposed engine binds 0.0.0.0, but the Host-header (DNS-rebinding) guard defaults to
        # loopback-only when RALLY_ALLOWED_HOSTS is unset (see main._allowed_hosts_from_env) ? it would
        # then 400 every request the edge forwards with its public Host. Boot WARN beats a silent 400-all.
```

```
    if not (env.get("RALLY_ALLOWED_HOSTS", "") or "").strip():
        out.append(
            StartupCheck(
                LEVEL_WARN,
                "EXPOSED_HOST_ALLOWLIST_LOOPBACK",
                "security.edge_auth_enabled is ON but RALLY_ALLOWED_HOSTS is unset ? the Host-header guard "
                "defaults to loopback-only and will 400 forwarded requests. Set RALLY_ALLOWED_HOSTS to your "
                "public host(s) (or '*' when the edge gate validates the host).",
            )
        )

    return out


def run_startup_checks(
    config: Any,
    *,
    cuda_available: Optional[bool] = None,
    env: Optional[Mapping[str, str]] = None,
    include_exposure: bool = False,
) -> list[StartupCheck]:
    """Return the startup findings for ``config``.

    ``cuda_available`` is the caller's best-effort GPU probe (``None`` = not probed ? GPU checks
    are skipped; the API server passes ``None``, the worker passes ``presets.probe_cuda_available()``).
    ``include_exposure`` adds the exposure-safety posture (``_exposure_checks``) ? the HTTP **server**
    passes ``True``; the worker leaves it ``False`` (it is never "exposed"). Both dimensions are
    DEFAULT-OFF: no profile ? no profile checks; ``edge_auth_enabled=False`` ? no exposure checks.
    Pure: no IO (beyond the ``find_spec`` auth probe), no torch import ? the caller owns the GPU probe.
    """
    e = os.environ if env is None else env
    checks: list[StartupCheck] = []

    # Exposure-safety posture ? INDEPENDENT of deployment.profile (a tunnel-to-a-box exposure often
    # runs with no profile set), gated on edge_auth_enabled (default-OFF). Server-only.
    if include_exposure:
        checks.extend(_exposure_checks(config, e))

    profile = getattr(getattr(config, "deployment", None), "profile", "") or ""
    if not profile:
        return checks  # no profile ? only the exposure posture (itself off unless edge auth is on).

    storage_backend = getattr(getattr(config, "storage", None), "backend", "local")
    indexing = getattr(config, "indexing", None)
    detector_impl = getattr(indexing, "detector_impl", "auto")
    device = getattr(getattr(indexing, "wasb", None), "device", "cuda")

    if profile == "truly-local":
        # truly-local reads bytes from the local FS or a *mounted* Drive ? a cloud backend here is
        # an inconsistent (but not fatal) choice worth surfacing.
        if storage_backend not in ("local", "gdrive"):
            checks.append(
                StartupCheck(
                    LEVEL_WARN,
```

```
                        "STORAGE_UNUSUAL",
                        f"profile 'truly-local' usually uses local/gdrive storage, but storage.backend="
                        f"'{storage_backend}'. Reads will go to a remote backend.",
                    )
                )
            # GPU heads-up (warning only ? the detector healthcheck is the authority; the probe is
            # best-effort and may be False even when a subprocess detector env has CUDA).
            if cuda_available is False:
                if device == "cuda":
                    checks.append(
                        StartupCheck(
                            LEVEL_WARN,
                            "CUDA_NOT_VISIBLE",
                            "device='cuda' but no CUDA GPU is visible to this process. If the box truly "
                            "has no GPU the detector healthcheck will hard-fail ? set indexing.wasb.device="
                            "'auto' for the CPU fallback (M4).",
                        )
                    )
                elif device == "auto":
                    checks.append(
                        StartupCheck(
                            LEVEL_WARN,
                            "CPU_FALLBACK",
                            "device='auto' and no CUDA GPU is visible to THIS process ? auto may resolve "
                            "to CPU (much slower). If torch lives only in the detector subprocess env this "
                            "can be a false alarm; the runner's DeviceContext/healthcheck is the authority.",
                        )
                    )

        elif profile == "cloud-serving":
            # The ONE fatal coherence check: a cloud profile that can't reach a bucket can't serve.
            if storage_backend not in _CLOUD_STORAGE:
                checks.append(
                    StartupCheck(
                        LEVEL_ERROR,
                        "STORAGE_INCOHERENT",
                        f"profile 'cloud-serving' requires a cloud storage backend "
                        f"({'/'.join(_CLOUD_STORAGE)}), but storage.backend='{storage_backend}'. "
                        "A cloud job cannot read/write a local-only backend.",
                    )
                )
            elif storage_backend == "gcs" and not _creds_discoverable(e):
                checks.append(
                    StartupCheck(
                        LEVEL_WARN,
                        "CREDS_UNCONFIRMED",
                        "no GOOGLE_APPLICATION_CREDENTIALS / Cloud Run env detected; relying on Application "
                        "Default Credentials. The GCS backend will fail loudly at init if ADC can't resolve.",
                    )
                )
            if device == "cpu":
                checks.append(
                    StartupCheck(
                        LEVEL_WARN,
```

```
                    "CLOUD_CPU_DEVICE",
                    "profile 'cloud-serving' runs on a GPU (L4) but indexing.wasb.device='cpu' ? "
                    "detection will not use the GPU.",
                )
            )
        elif cuda_available is False and device in ("cuda", "auto"):
            checks.append(
                StartupCheck(
                    LEVEL_WARN,
                    "CLOUD_CUDA_NOT_VISIBLE",
                    "profile 'cloud-serving' expects a GPU but no CUDA GPU is visible to this process.",
                )
            )

    elif profile == "cpu-mock":
        # The mock profile is for CI/regression ? a real detector here defeats the purpose.
        if detector_impl != "stub":
            checks.append(
                StartupCheck(
                    LEVEL_WARN,
                    "MOCK_NONSTUB_DETECTOR",
                    f"profile 'cpu-mock' normally uses detector_impl='stub', but detector_impl="
                    f"'{detector_impl}'. A real detector defeats the GPU-free mock.",
                )
            )

    return checks


def enforce_startup_checks(
    config: Any,
    *,
    cuda_available: Optional[bool] = None,
    env: Optional[Mapping[str, str]] = None,
    raise_on_error: bool = True,
    include_exposure: bool = False,
    logger: Optional[logging.Logger] = None,
) -> list[StartupCheck]:
    """Run the checks, LOG each (errors at ERROR, warnings at WARNING), and ? unless
    ``raise_on_error=False`` ? raise ``StartupCheckError`` if any FATAL check tripped.

    ``include_exposure`` (server-only) adds the exposure-safety posture. Returns the full findings
    list (so a caller can inspect/surface them). A no-op with no logging and no raise when both
    dimensions are off (``run_startup_checks`` returns ``[]``)."""
    log = logger or logging.getLogger(__name__)
    checks = run_startup_checks(
        config,
        cuda_available=cuda_available,
        env=env,
        include_exposure=include_exposure,
    )
    for c in checks:
        if c.level == LEVEL_ERROR:
            log.error("[STARTUP] %s: %s", c.code, c.message)
        else:
            log.warning("[STARTUP] %s: %s", c.code, c.message)
    errors = [c for c in checks if c.level == LEVEL_ERROR]
    if errors and raise_on_error:
        raise StartupCheckError(errors)
```

```
        return checks
```

10. user (2026-07-02T02:48:07.196Z)

OPEN | Allow gs:// / s3:// / gdrive-api:// ingest on an exposed (jailed) box ? scheme-exempt recognized storage backends from the path-jail

Today

When `security.allowed_video_root` is set (required to expose), `/api/process` runs `validate_input_path` (local-under-jail) **before** resolving the storage scheme, so `gs://bucket/x.mp4` ? `os.path.realpath` escapes the jail ? **400**. Bucket ingest is therefore impossible on the exposed alpha box. Pinned by `tests/test_api_endpoints.py::test_process_hosted_mode_rejects_nonlocal_uri` (gs/s3/gdrive all 400).

Want

The owner wants to ingest large videos via `gs://` on the hosted box (far more convenient than a 500 MB browser upload).

Proposed

Make `validate_input_path` / the `/api/process` preamble **scheme-aware**: either (a) allow recognized storage schemes (`gs://`, `s3://`, `gdrive-api://`) *before* the local-jail check, or (b) apply the jail **only to local refs** (validate after `storage.resolve_input_ref`, jailing only `backend == "local"`).

? Must NOT reopen the arbitrary-local-file hole ? the jail must still fully apply to **local** paths (traversal/symlink/absolute). Add tests for both the allow (bucket URI accepted) and the reject (local traversal still 400) paths. Pairs with the `gs://` ingest the alpha CUJ + DEPLOY_ALPHA §7 currently disclaim.

11. user (2026-07-02T02:48:08.152Z)

```
backend\main.py:39:from fastapi.middleware.cors import CORSMiddleware
backend\main.py:43:from starlette.middleware.trustedhost import TrustedHostMiddleware
backend\main.py:110:    """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-separated),
backend\main.py:114:    raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
backend\main.py:119:# CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by browsers per the
backend\main.py:124:    CORSMiddleware,
backend\main.py:125:    allow_origins=_cors_origins_from_env(),
backend\main.py:132:def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
backend\main.py:143:    # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's TrustedHostMiddleware
backend\main.py:153:    TrustedHostMiddleware,
backend\main.py:154:    allowed_hosts=_allowed_hosts_from_env(),
backend\config\startup.py:155:    # loopback-only when RALLY_ALLOWED_HOSTS is unset (see main._allowed_hosts_from_env) ? it would
backend\config\models.py:355:    # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at import in
```

12. user (2026-07-02T02:48:13.796Z)

```
        ``request.app.state`` exclusively."""
        global global_config, db_instance
        global_config = config
        db_instance = db
        app.state.config = config
        app.state.db = db
        return app
```

```python
def _cors_origins_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
    """M9: CORS allow-origins from the ``RALLY_CORS_ALLOW_ORIGINS`` env var (comma-separated),
    default ``["*"]``. Read from the ENV (not a cwd ``config.json``) so importing ``backend.main``
    does zero filesystem I/O ? a hosted/multi-tenant deploy sets this to lock CORS to its origin."""
    src = os.environ if env is None else env
    raw = (src.get("RALLY_CORS_ALLOW_ORIGINS", "") or "").strip()
    origins = [o.strip() for o in raw.split(",") if o.strip()]
    return origins or ["*"]


# CORS for the web UI. allow_origins='*' with allow_credentials=True is rejected by browsers per the
# fetch spec + is unsafe to carry into multi-tenant; this tool uses no cookies, so credentials stay
# off. The origin list is configurable via the env var above (default ["*"]) so a hosted deploy can
# lock it to its origin; the middleware is fixed at startup.
app.add_middleware(
    CORSMiddleware,
    allow_origins=_cors_origins_from_env(),
    allow_credentials=False,
    allow_methods=["*"],
    allow_headers=["*"],
)


def _allowed_hosts_from_env(env: Optional[Dict[str, str]] = None) -> List[str]:
    """P0c: Host-header allowlist ? a DNS-rebinding guard for the localhost appliance. A malicious
    web page can point a domain it controls at 127.0.0.1 and script requests to the local server;
    the browser sends ``Host: attacker.com``, which this rejects (the server only answers loopback
    Host headers by default). ``RALLY_ALLOWED_HOSTS`` (comma-separated) overrides; the default keeps
    Starlette's TestClient host ``testserver`` so the in-process suite isn't broken. Read from the
    ENV (not config.json) so importing ``backend.main`` does zero filesystem I/O; a routable/edge
    deploy sets its public host (or ``*`` when fronted by an auth gate that validates the host)."""
    src = os.environ if env is None else env
    raw = (src.get("RALLY_ALLOWED_HOSTS", "") or "").strip()
    hosts = [h.strip() for h in raw.split(",") if h.strip()]
    # IPv4 loopback (the default bind) + the TestClient host. NB: Starlette's TrustedHostMiddleware
    # matches on host.split(":")[0], which mangles a bracketed IPv6 Host ("[::1]:port"), so IPv6
    # loopback is NOT supported by this guard ? access the appliance via 127.0.0.1/localhost.
    return hosts or ["localhost", "127.0.0.1", "testserver"]


# DNS-rebinding guard. Bound to localhost by default, so only loopback Host headers are honoured
# (a remote page can't read responses cross-origin AND its Host header is rejected here). www_redirect
# off so an unexpected ``www.`` host is rejected rather than 307-redirected.
app.add_middleware(
    TrustedHostMiddleware,
    allowed_hosts=_allowed_hosts_from_env(),
    www_redirect=False,
```

```
    )


def _resolve_bind_host(
    config: Optional[AppConfig] = None, cli_host: Optional[str] = None
) -> str:
    """P0c: the server bind address. Precedence: ``--host`` > ``RALLY_BIND_HOST`` env > ``0.0.0.0``
    when an edge-auth profile is active (``security.edge_auth_enabled`` ? i.e. there IS an auth gate
    in front) > ``127.0.0.1`` (default: localhost-only, today's behaviour). Binding a routable
    interface with no auth gate is unsafe, hence the localhost default; widen it consciously."""
    if cli_host and cli_host.strip():
        return cli_host.strip()
    env = os.environ.get("RALLY_BIND_HOST")
    if env and env.strip():
```

13. user (2026-07-02T02:48:14.151Z)

```
# PEP 621 packaging for the badminton/sports highlight indexer (DEFERRED #7).
#
# Build backend: hatchling. `backend/` is an intentional NAMESPACE package
# (no `backend/__init__.py`; mypy.ini sets explicit_package_bases for the same
# reason), so the wheel target packages are listed EXPLICITLY below ? hatchling's
# auto-detection would otherwise miss the un-__init__'d `backend` root.
#
# torch/torchvision are deliberately NOT listed in [project.dependencies]:
# their CPU/CUDA wheels need an out-of-band index (--index-url / --extra-index-url),
# which PEP 621 cannot express. Install them separately ? see [project.optional-dependencies]
# `gpu` (documentation only) and the README. CI installs CPU torch on its own line.
[build-system]
requires = ["hatchling"]
build-backend = "hatchling.build"

[project]
name = "badminton-highlight-indexer"
version = "0.1.0"
description = "AI-assisted rally segmentation and highlight indexing for racket sports."
readme = "README.md"
requires-python = ">=3.10"
license = { text = "MIT" }
authors = [{ name = "Avi Dullu", email = "avi.dullu@gmail.com" }]
keywords = ["badminton", "sports", "video", "highlights", "rally", "segmentation"]

# Runtime deps mirror requirements.txt, EXCEPT torch/torchvision (see header).
dependencies = [
    "fastapi>=0.111.0",
    "uvicorn>=0.30.1",
    "opencv-python>=4.9.0.80",
    "numpy>=1.26.4",
    "pydantic>=2.7.1",
    "jinja2>=3.1.4",
    "python-dotenv>=1.0.0",
    "google-genai>=2.4.0",
    "supervision>=0.21.0",
    # Used directly on the serving path (backend/features/rally_seq.py: uniform_filter1d /
    # maximum_filter1d); previously present only TRANSITIVELY via supervision, so a supervision
    # bump or a pruned-transitive freeze could silently break inference. Declare it. (BSD-3.)
    "scipy>=1.10",
```

```
    # D13/P12: ByteTrack moved to the standalone `trackers` package (Apache-2.0). Its
    # PyPI wheel is broken (metadata-only), so it MUST be installed from git ? see
    # requirements.txt. PEP 621 cannot express a git URL, so it's NOT listed here
    # (same pattern as torch/torchvision). CI installs it on its own line.
]

[project.optional-dependencies]
# Test + lint/type tooling ? matches the CI lanes (.github/workflows/ci.yml).
dev = [
    "pytest",
    "pytest-cov",
    "httpx",        # fastapi TestClient transport
    "ruff",
    "mypy",
    "PyJWT[crypto]>=2.8",  # M9 edge-auth: Cf-Access JWT verify (RS256) ? tests + CI
]

# M9 edge-auth (Cloudflare Access). DEFAULT-OFF: only needed when security.edge_auth_enabled=True.
# `jwt` is lazy-imported, so the base runtime works without this; install it for hosted/multi-tenant.
auth = ["PyJWT[crypto]>=2.8"]

# Per-video observability reports (SOFT dependency: the pipeline degrades to a
# no-op if absent). Pinned to its git tag per BACKLOG. The private repo is
# fetched over SSH; if you lack access, simply omit this group ? reporting
# tests skip via pytest.importorskip. For local dev against a sibling checkout:
#   pip install -e ../sports-obsreport --no-build-isolation
reporting = [
    "obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3",
]

# `gpu` is DOCUMENTATION ONLY ? it is intentionally empty. torch/torchvision
# CUDA wheels require `--index-url`/`--extra-index-url`, which PEP 621 cannot
# express, so they are NOT declarable here. Install them out-of-band instead:
#   # CUDA 12.4 box:
#   pip install torch>=2.12.0 torchvision>=0.27.0 --index-url https://download.pytorch.org/whl/cu124
#   # CPU-only box (also what CI uses):
#   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
gpu = []

# Storage backends (docs/DECOUPLED_COMPUTE/). The default `local` backend needs NONE of
# these. Cloud SDKs are imported lazily, so `import backend.storage` works without them;
# install the extra for the bucket you use. (boto3's S3 client covers AWS + Cloudflare R2 +
# DO Spaces + MinIO via endpoint_url.) `gdrive://` needs NO extra ? it reads a locally
# MOUNTED Google Drive (Google Drive for Desktop) as plain files; see backend/storage/gdrive.py.
storage-s3 = ["boto3>=1.34"]
storage-gcs = ["google-cloud-storage>=2.16"]

[project.scripts]
# `indexer --server`, `indexer --help`, etc. -> backend/main.py::main()
indexer = "backend.main:main"

[tool.hatch.metadata]
# The `reporting` extra pins obsreport to a git tag (a PEP 508 *direct
# reference*); hatchling requires opting in to allow those in metadata.
allow-direct-references = true

[tool.hatch.build.targets.wheel]
# `backend` is a namespace package (no __init__.py); list both import roots
# explicitly so an editable/wheel install exposes `import backend` and
# `import training` regardless of cwd.
```

```
    packages = ["backend", "training"]

```

14. user (2026-07-02T02:48:30.568Z)

```
README.md:20:[Omitted long matching line]
.github\workflows\nightly.yml:165:  # copyleft (GPL/AGPL/LGPL/SSPL). This is exactly the
transitive-dep drift the
.github\workflows\nightly.yml:183:      - name: Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the
runtime closure
.github\workflows\nightly.yml:186:            echo "::error::A runtime dependency carries a
copyleft (GPL/AGPL/LGPL/SSPL) license ? the guardrail is MIT/Apache/BSD only. Review the report
above."
.github\workflows\nightly.yml:189:          echo "OK: no GPL/AGPL/LGPL/SSPL in the runtime
dependency closure."
.github\workflows\ci.yml:178:  # (GPL/AGPL/LGPL/SSPL). torch (BSD-3) is installed out-of-band
and is not in this closure. This
.github\workflows\ci.yml:199:      - name: Fail on any copyleft (GPL/AGPL/LGPL/SSPL) in the
runtime closure
.github\workflows\ci.yml:202:            echo "::error::A runtime dependency carries a copyleft
(GPL/AGPL/LGPL/SSPL) license ? the guardrail is MIT/Apache/BSD only. Review the report above."
.github\workflows\ci.yml:205:          echo "OK: no GPL/AGPL/LGPL/SSPL in the runtime dependency
closure."
docs\archives\research\AUDIO_RALLY_DETECTION_PLAN.md:69:  numpy STFT, so the originally-
anticipated scipy dependency was not added. Avoid heavy/AGPL audio stacks. Any
docs\archives\decisions\DECISIONS.md:239:[Omitted long matching line]
docs\archives\decisions\DECISIONS.md:243:[Omitted long matching line]
docs\archives\decisions\DECISIONS.md:251:- Detector swap is contained to
`backend/indexer/yolo_hybrid.py` (the `from ultralytics import YOLO` load + the
`model.track(...)` call) plus `config.json` knobs and `tests/test_yolo_hybrid.py` mocks. Stage-1
feature outputs (velocity/active-frame/track-id stats) and Stages 2?4 are detector-agnostic and
unchanged.
docs\archives\decisions\DECISIONS.md:252:- We now own the frame loop + tracking (ultralytics did
both internally). Use a permissive ByteTrack (e.g. `supervision`, MIT, or the MIT ByteTrack
repo) on top of `lapx` (already a BSD dep).
docs\archives\decisions\DECISIONS.md:253:- Residual items needing legal sign-off: exact codec-
pool royalty thresholds (AVC/HEVC decode of uploads), Ultralytics Enterprise pricing, and per-
checkpoint weight licenses ? tracked in `docs/MONETIZATION_AUDIT.md` §7.
docs\archives\MONETIZATION_AUDIT.md:7:> forward. (Notably: the AGPL/Ultralytics blocker in §1 is
now resolved via ADR-005.)
docs\archives\MONETIZATION_AUDIT.md:12:**Key legal premise (verified):** We do **not**
distribute code or binaries to users; everything runs server-side. Therefore *distribution-
triggered* copyleft (GPL/LGPL) is **not** triggered. BUT **AGPL §13** (network/SaaS clause) and
**external metered-API terms** still apply, and **codec patent royalties** are a separate axis
from software licenses.
docs\archives\MONETIZATION_AUDIT.md:22:| **ultralytics** (lib) | **AGPL-3.0** | **Yes** ? §13
network clause; importing the lib into our app + serving over a network = source-disclosure of
the *whole* app | ? **BLOCKER** | Replace (see §2) or buy Enterprise License |
docs\archives\MONETIZATION_AUDIT.md:23:| **`yolov8n.pt`** weights | **AGPL-3.0** (same as code;
HF card tagged `agpl-3.0`) | **Yes** | ? **BLOCKER** | Replace weights; swapping codebase but
keeping `.pt` does *not* escape |
docs\archives\MONETIZATION_AUDIT.md:28:| **ffmpeg / ffprobe** | LGPL-2.1+ (GPL if `--enable-
gpl`); **not AGPL** | **No** ? no distribution, no network clause | ? CLEAR | Keep; invoke as
subprocess (already do) |
docs\archives\MONETIZATION_AUDIT.md:36:**Bottom line:** Only **two** real blockers ? (1)
**Ultralytics** (AGPL, code *and* weights) and (2) two specific **model weights** (MonoTrack,
YOLO-NAS) we must simply avoid. Everything else is either already clean or a manageable WATCH.
docs\archives\MONETIZATION_AUDIT.md:44:### Player / motion-gating detector (replaces
`ultralytics` YOLOv8n)
docs\archives\MONETIZATION_AUDIT.md:51:| **RT-DETRv2** (`lyuwenyu`/Baidu) | **Apache-2.0** |
Real-time, NMS-free transformer; competitive with YOLO. Use the **Baidu** repo, *not* the
ultralytics RT-DETR port (that inherits AGPL). |
docs\archives\MONETIZATION_AUDIT.md:52:| **D-FINE** | **Apache-2.0** core + weights | ICLR'25
SOTA real-time. ?? repo bundles an **AGPL YOLO** subcomponent ? use only the Apache RT-DETR-
derived core. |
```

docs\archives\MONETIZATION_AUDIT.md:71:- **Ultralytics Enterprise License** ? removes AGPL for closed-source/SaaS use of code *and* models. Pricing is **quote-only / not public** (confirmed via Ultralytics GH issues #15877, #1888, #9853 ? all routed to sales) ??. Treat as a material recurring cost; only sensible if YOLOv8 accuracy proves irreplaceable.
docs\archives\MONETIZATION_AUDIT.md:110:1. **Ultralytics Enterprise price** ? get a quote to compare against swap engineering cost.
docs\archives\MONETIZATION_AUDIT.md:118:- Ultralytics License (AGPL-3.0 + Enterprise): `ultralytics.com/license`, `docs.ultralytics.com/help/license/`, repo `LICENSE` (GitHub API `spdx_id=AGPL-3.0`), weights tag on HF `Ultralytics/YOLOv8` (`agpl-3.0`), GH issues #15877/#1888/#9853.
docs\archives\MONETIZATION_AUDIT.md:119:- AGPL-3.0 §13 text: `gnu.org/licenses/agpl-3.0`.
docs\archives\MONETIZATION_AUDIT.md:123:- Detectors: `Megvii-BaseDetection/YOLOX` (Apache), `lyuwenyu/RT-DETR` (Apache), `Peterande/D-FINE` (Apache + AGPL YOLO caveat), `Deci-AI/super-gradients` `LICENSE.YOLONAS.md` (noncommercial weights).
docs\ROADMAP.md:25:> AGPL YOLOv8 was removed and replaced with a BSD/MIT-licensed torchvision +
docs\ROADMAP.md:27:> player-motion gating stage, not the Ultralytics library.
docs\MULTI_SIGNAL_FUSION_PLAN.md:231:Every dependency stays **MIT / Apache-2.0 / BSD** (ADR-002 / ADR-005; `ultralytics` was dropped for
docs\MULTI_SIGNAL_FUSION_PLAN.md:232:AGPL-3.0).
docs\MULTI_SIGNAL_FUSION_PLAN.md:241:| **Banned** | **Ultralytics YOLOv5/8/11 (AGPL-3.0)** · **MonoTrack (Adobe, non-comm)** · **YOLO-NAS (Deci, non-comm)** | ? | ? |
docs\MULTI_SIGNAL_FUSION_PLAN.md:297: (no `ultralytics` import; detector factories stay torchvision/Apache).
docs\PACKAGING_PLAN.md:55:- **New packaging deps that clear L6 `[researched]`:** `uv` (Apache/MIT), Inno Setup (permissive), `onnxruntime` (MIT), `micromamba` (BSD-3), Docker/OCI (Apache-2.0), boto3/google-cloud-* (Apache-2.0), stdlib `webbrowser`. **Forbidden:** `pystray` (LGPL-3.0). **Build-time-only:** `Nuitka` 4.x is **AGPL-3.0+** (compiled output has a runtime exception) ? usable as a tool, never bundled. `obsreport` (private git dep) stays a truly-optional extra, never bundled.
docs\PACKAGING_PLAN.md:103:| P0d | **Declare `scipy`** in `[project.dependencies]` (serving use `[verified features/rally_seq.py:17]`, was transitive-only) + **license-scan CI lane** (pip-licenses; fails on GPL/AGPL/LGPL/SSPL) + **built-wheel smoke lane** (build ? install ? `indexer --help` from a non-repo cwd). Both lanes green on CI. | ? | No | ? | [#297](https://github.com/Khelsutra/badminton-highlight-indexer/pull/297) |
docs\OWNED_MODEL_IMPLEMENTATION_PLAN.md:38: GRPO/RFT with a custom video reward** in one stack. Keep **unsloth** (Apache-2.0 core; *avoid its AGPL Studio UI*)
docs\CODE_MAP.md:161:[Omitted long matching line]
docs\BACKLOG.md:31:- ? **Ultralytics Enterprise pricing** ? get a quote **only if** the torchvision/ByteTrack detector (PR #8) proves insufficient on real footage. Currently *not needed* (swap done). Quote-only pricing. **Revisit:** if detector quality forces reconsidering YOLOv8.
docs\COMMERCIALIZATION.md:22:**AGPL §13 (network clause)**, **metered-API terms**, and **video-codec patent
docs\COMMERCIALIZATION.md:50:| C1 | **AGPL / copyleft removal** | ? | ? | ADR-005 shipped: AGPL `ultralytics` YOLOv8 replaced with **torchvision Faster R-CNN (BSD) + `supervision` ByteTrack (MIT)**. No AGPL code/weights in the stack. | Done. Keep new deps permissive. |
docs\COMMERCIALIZATION.md:62:| ~~`ultralytics` + `yolov8n.pt`~~ | AGPL-3.0 | Yes (§13) | ? **REMOVED** | Replaced per ADR-005 ? no longer in the stack. |
docs\COMMERCIALIZATION.md:72:**Bottom line:** the two hard blockers from the audit (Ultralytics; banned weights)
docs\COMMERCIALIZATION.md:190:1. **Ultralytics Enterprise price** ? only relevant if a permissive detector ever proves
docs\COMMERCIALIZATION.md:208: audit + the commercial-readiness review. **Refreshed since the audit:** C1 AGPL
backend\pipeline\detectors\person_detector.py:28:(ADR-005; ultralytics' AGPL YOLO was dropped).
backend\pipeline\detectors\person_detector.py:41:# class 1 (index 0 is the implicit background). NB: ultralytics YOLO used class 0 ?
backend\pipeline\detectors\person_detector.py:140:   This replaces the old ultralytics ``model.track(...)`` call, which bundled
backend\pipeline\segmenters\yolo_hybrid.py:9:Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted

15. user (2026-07-02T02:48:31.105Z)

1   fastapi>=0.111.0

```
uvicorn>=0.30.1
opencv-python>=4.9.0.80
numpy>=1.26.4
pydantic>=2.7.1
jinja2>=3.1.4
python-dotenv>=1.0.0
google-genai>=2.4.0
# torch/torchvision are listed here but deliberately NOT in pyproject.toml
# [project.dependencies]. Their CPU/CUDA wheels need an out-of-band index
# (--index-url / --extra-index-url) that PEP 621 cannot express. Install
# them separately matching your hardware:
#   NVIDIA GPU: pip install torch torchvision --index-url https://download.pytorch.org/whl/cu124
#   CPU only:   pip install torch torchvision --index-url https://download.pytorch.org/whl/cpu
# CI installs CPU torch on its own line in .github/workflows/ci.yml.
torch>=2.12.0
torchvision>=0.27.0
supervision>=0.21.0
# D13/P12: ByteTrack moved out of `supervision` (removed in 0.30.0) to the standalone
# `trackers` package. The PyPI wheel is BROKEN (ships metadata only, no source ? see
# https://github.com/roboflow/trackers/issues/474), so install from git until fixed:
#   pip install "trackers @ git+https://github.com/roboflow/trackers.git@2.5.0"
# (Apache-2.0; depends on supervision >=0.26.1, so it composes cleanly.)
# NOTE: roboflow/trackers uses bare version tags (2.5.0), NOT v-prefixed.
# PIN-DUPLICATION (mirrors the torch carve-out): this exact ref is ALSO hard-coded in
# .github/workflows/ci.yml + nightly.yml (full-suite lanes) and deploy/digitalocean/bootstrap.sh
# + deploy/cloudrun/Dockerfile ? edit ALL of them in lockstep on a bump, or the lanes/boxes
# silently resolve different versions.
trackers @ git+https://github.com/roboflow/trackers.git@2.5.0
scipy>=1.10

# Per-video observability reports (SOFT dependency: the pipeline degrades to a
# no-op if absent). Shared, permissively-licensed report toolkit. Pin a tag once
# the private repo is published, e.g.:
#   obsreport @ git+ssh://git@github.com/avidullu/sports-obsreport.git@v0.1.3
# For local development install it editable from a sibling checkout:
#   pip install -e ../sports-obsreport --no-build-isolation

```

16. user (2026-07-02T02:48:47.007Z)

```
.gitignore: Unicode text, UTF-8 text, with CRLF line terminators
8:.env
9:*.env
10:.env.*
11:# ...but deploy/gcp/buckets.env is the TRACKED single-source-of-truth for PUBLIC project + bucket
13:# MUST be committed. Keep real secret .env files ignored above.
14:!deploy/gcp/buckets.env
52:# Local generated artifacts (review reels, eval CSVs, e2e/proof outputs) -- regenerable, never committed
53:artifacts/
69:# Throwaway repro scripts + PR-body / handoff scratch notes (already excluded from
71:scratch/
---tracked artifacts/scratch---
scratch/analyze_motion.py
scratch/clean_state.py
scratch/diagnose_rally.py
scratch/gemini_video_debugger.py
scratch/list_models.py
scratch/test_ai_segmenter.py
scratch/test_thresholds.py
```

```
---tracked env/config---
config.json
---untracked status---
!! artifacts/
!! scratch/ABLATION_FRAMEWORK_PLAN.md
!! scratch/GATE_AB_NUMBERS_AND_IMPLICATIONS.md
!! scratch/HIGH_RISK_REMEDIATION_PLAN.md
!! scratch/OWNED_MODEL_PHASE0_PLAN.md
!! scratch/PACKAGING_PLAN.serving-wip.md
!! scratch/PERSONALIZATION_FEATURE_DESIGN.md
!! scratch/PERSONALIZATION_PIVOT_ANALYSIS.md
!! scratch/PR1_BODY.md
!! scratch/PR1_HANDOFF.md
```

17. user (2026-07-02T02:48:47.412Z)

```
deploy\digitalocean\gpu_setup.sh
deploy\digitalocean\mount_scratch.sh
deploy\digitalocean\bootstrap.sh
deploy\digitalocean\README.md
```

18. user (2026-07-02T02:49:08.782Z)


```
{
 "sport": "badminton",
 "ai_provider": "gemini",
 "ai_model": "gemini-flash-latest",
 "validation": {
  "min_resolution_width": 1280,
  "min_resolution_height": 720,
  "min_fps": 29.9,
  "min_blur_threshold": 20.0,
  "max_scene_cuts_per_minute": 0.5,
  "relevance": {
   "court_green_lower": [
    35,
    40,
    40
   ],
   "court_green_upper": [
    85,
    255,
    255
   ],
   "court_pixels_min_ratio": 0.05
  }
 },
 "indexing": {
  "default_segmenter": "gemini",
  "use_gpu": true,
  "motion_threshold": 0.08,
  "min_segment_duration": 3.0,
  "dead_time_merge_gap": 2.0,
  "chunk_duration": 90,
  "chunk_overlap": 10,
  "detector_model": "fasterrcnn_resnet50_fpn",
  "detector_score_threshold": 0.5,
  "yolo_velocity_threshold": 0.15,
  "yolo_frame_skip": 3,
  "yolo_tracker": "bytetrack.yaml",
  "yolo_prefetch_workers": 2,
  "min_action_window_duration": 2.0,
  "log_yolo_candidates": true,
  "store_yolo_candidates": true,
  "ai_handoff_padding": 2.0,
```

```
  "ai_max_workers": 1,
  "tracknet": {
   "wsl_distro": "Ubuntu",
   "repo_dir": "~/models/TrackNetV4",
   "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
   "conda_env": "TrackNetV4",
   "weights_path": "",
   "queue_length": 5,
   "stage_in_wsl": true,
   "wsl_stage_dir": "~/clips",
   "timeout_sec": 1800,
   "velocity_threshold": 0.02
  },
  "wasb": {
   "wsl_distro": "Ubuntu",
   "repo_dir": "~/models/WASB-SBDT",
   "conda_sh": "~/miniconda3/etc/profile.d/conda.sh",
   "conda_env": "wasb",
   "weights_path": "~/models/WASB-SBDT/pretrained_weights/wasb_badminton_best.pth.tar",
   "sport": "badminton",
   "wsl_stage_dir": "~/clips",
   "timeout_sec": 1800,
   "velocity_threshold": 0.02
  }
 },
 "output": {
  "default_highlights_name": "highlights.mp4",
  "db_name": "sports_indexer.db"
 },
 "stitching": {
  "begin_padding": 0.5,
  "end_padding": 0.5,
  "use_cache": false,
  "min_shot_count": 1,
  "watermark": {
   "enabled": true,
   "text": "Generated by Khelsutra.guru",
   "position": "bottom-right",
   "opacity": 0.85,
   "logo_path": "",
   "font_path": ""
  }
 }
}
```


19. user (2026-07-02T02:49:10.641Z)

```
.github\workflows\nightly.yml:131:      - name: Install trackers (git-only dep ? PyPI wheel is broken)
.github\workflows\nightly.yml:132:        # D13/P12: ByteTrack moved to the standalone `trackers` package (git-only;
.github\workflows\nightly.yml:135:        run: python -m pip install "trackers @ git+https://github.com/roboflow/trackers.git@2.5.0"
.github\workflows\ci.yml:124:      - name: Install trackers (git-only dep ? PyPI wheel is broken)
.github\workflows\ci.yml:127:        # source ? roboflow/trackers#474), so it MUST come from git. PEP 621 can't
.github\workflows\ci.yml:131:        # NOTE: roboflow/trackers uses bare version tags (2.5.0), NOT v-prefixed.
.github\workflows\ci.yml:132:        run: python -m pip install "trackers @ git+https://github.com/roboflow/trackers.git@2.5.0"
.github\workflows\ci.yml:180:  # D13/P12 carve-out: `trackers` (Apache-2.0; ByteTrack's new home) is also git-only and thus
requirements.txt:21:# https://github.com/roboflow/trackers/issues/474), so install from git until fixed:
```

```
requirements.txt:22:#   pip install "trackers @
git+https://github.com/roboflow/trackers.git@2.5.0"
requirements.txt:24:# NOTE: roboflow/trackers uses bare version tags (2.5.0), NOT v-prefixed.
requirements.txt:29:trackers @ git+https://github.com/roboflow/trackers.git@2.5.0
deploy\digitalocean\bootstrap.sh:42:# `trackers` package. Git-only dep (PyPI wheel is metadata-
only ? roboflow/trackers#474),
deploy\digitalocean\bootstrap.sh:46:pip install "trackers @
git+https://github.com/roboflow/trackers.git@2.5.0"
deploy\cloudrun\Dockerfile:58:# D13/P12: `trackers` (ByteTrack's new home) is a git-only dep
(broken PyPI wheel ?
deploy\cloudrun\Dockerfile:59:# roboflow/trackers#474) so it's NOT pulled by `pip install .`;
install it out-of-band before
deploy\cloudrun\Dockerfile:62:    && python3 -m pip install "trackers @
git+https://github.com/roboflow/trackers.git@2.5.0" \
backend\ui\assets\index-ukuS9wxa.js:60:[Omitted long matching line]
docs\CODE_CLEANUP_AUDIT_2026-06-24.md:200:| **P12** | **D13: ByteTrack deprecation** ? migrated
`sv.ByteTrack` ? `trackers.ByteTrackTracker` (git dep; PyPI wheel broken) | P7 | No | ? |
[#386](https://github.com/Khelsutra/badminton-highlight-indexer/pull/386) |
docs\CODE_CLEANUP_AUDIT_2026-06-24.md:246:| ~~4~~ | ~~**D13 / P12**~~ | ~~ByteTrack deprecation
migration~~ ? **? DONE.** `sv.ByteTrack` ? `trackers.ByteTrackTracker` (git dep; PyPI wheel
broken ? filed Roboflow issue). |
docs\CODE_CLEANUP_AUDIT_2026-06-24.md:282:[Omitted long matching line]
tests\test_person_detector.py:210:    Skipped when the `trackers` package isn't installed (it's
a git-only dep whose
```

20. user (2026-07-02T02:49:27.183Z)

Testing the rally pipeline on DigitalOcean

```
> **? DEPRECATED (ADR-013, 2026-06-20) ? DigitalOcean is DROPPED for compute.** Use
> **[`deploy/gcp/README.md`](../gcp/README.md)** ? GCP is the sole compute stack (DO GPU isn't
enabled
> on the account + isn't cleanly credit-funded; Paperspace is subscription-gated). Kept for
history.
> **NOTE:** the scripts here ? `bootstrap.sh`, `mount_scratch.sh`, `gpu_setup.sh` ? are
> **provider-agnostic** Linux-GPU setup and are **still reused by the GCP runbook**; only the
> DO-specific *provisioning* (doctl droplet create, Spaces) is retired. (Optional later cleanup:
move
> them to `deploy/common/`.)

A storage-free, secret-free way to get the pipeline running on a DigitalOcean (or Paperspace)
machine ? the **compute** half of [`docs/archives/past_projects/DECOUPLED_COMPUTE/`](../../docs/
archives/past_projects/DECOUPLED_COMPUTE/README.md).
The object-store **storage shim is intentionally out of scope here** (owner is finalizing it);
this
runbook only proves *compute* works on the box.

===
===
OPEN | ci: single-source the shared offline guard lanes + coverage floor via a workflow_call
reusable
OPEN | [cost] /api/process re-runs the paid segmenter for an already-processed video ?
refresh/"Try again" re-bills Gemini
OPEN | [safety] Content-safety pre-flight litmus: flag non-sports / NSFW video BEFORE sending it
to Gemini
```

21. user (2026-07-02T02:49:27.192Z)

```
"""Cloudflare Access edge-auth ? verify the Cf-Access JWT, derive the tenant ``user_id``
(M9, D9).

**DEFAULT-OFF:** when ``security.edge_auth_enabled`` is False the resolver is a strict
NO-OP (returns
``None`` = the local single-user) ? nothing about the existing local path changes. When
```

```
enabled, the
``Cf-Access-Jwt-Assertion`` header is **required** and **verified** (RS256) against the team JWKS ?
signature + exp + issuer + audience ? so a direct hit to the Cloud Run URL **can't spoof** the
identity header (the explicit D9 anti-spoof requirement). Returns the verified ``user_id`` (the
token's ``email``, else ``sub``).

``jwt`` (``PyJWT[crypto]``) is **lazy-imported** so the base runtime works without it (install the
``auth`` extra for hosted/multi-tenant). The JWKS fetch is **injectable**, so the whole verifier is
unit-tested **offline** with an in-test RSA keypair + a fake JWKS ? no network, no Cloudflare.
"""

from __future__ import annotations

import json
import time
import urllib.request
from typing import Any, Callable, Mapping, Optional

# Cloudflare Access sends the signed assertion in this header (it terminates at the CF edge).
CF_ACCESS_HEADER = "Cf-Access-Jwt-Assertion"
_JWKS_PATH = "/cdn-cgi/access/certs"
_JWKS_TTL_SEC = (
    600.0  # re-fetch the team JWKS at most every 10 min (keys rotate slowly)
)

JwksFetcher = Callable[[str], dict]


class EdgeAuthError(Exception):
    """A required Cf-Access token was missing or failed verification ? the caller returns 401/403."""


# module-level JWKS cache: team_domain -> (fetched_at, jwks_dict). Production only; tests inject.
_jwks_cache: dict[str, tuple[float, dict]] = {}


def _issuer(team_domain: str) -> str:
    return team_domain.rstrip("/")


def _default_jwks_fetcher(team_domain: str) -> dict:
    """Fetch the team's public JWKS (``<team_domain>/cdn-cgi/access/certs``). Network ? only the
    production path reaches here; tests pass an injected fetcher."""
    url = _issuer(team_domain) + _JWKS_PATH
    with urllib.request.urlopen(url, timeout=5) as resp:  # noqa: S310 (https team domain)
        data: Any = json.load(resp)
    if not isinstance(data, dict) or "keys" not in data:
        raise EdgeAuthError(f"unexpected JWKS shape from {url}")
    return data


def _get_jwks(team_domain: str, fetcher: Optional[JwksFetcher]) -> dict:
    """Return the JWKS for ``team_domain``, cached with a TTL. An injected ``fetcher`` (tests)
```

```
    bypasses the cache so each test is deterministic."""
    if fetcher is not None:
        return fetcher(team_domain)
    now = time.time()
    cached = _jwks_cache.get(team_domain)
    if cached and (now - cached[0]) < _JWKS_TTL_SEC:
        return cached[1]
    jwks = _default_jwks_fetcher(team_domain)
    _jwks_cache[team_domain] = (now, jwks)
    return jwks


def verify_cf_access_token(
    token: str, *, team_domain: str, aud: str, jwks: dict
) -> str:
    """Verify a Cf-Access JWT against ``jwks`` (RS256; checks sig + exp + issuer + audience) and
    return the user_id (``email`` else ``sub``). Raises ``EdgeAuthError`` on ANY failure."""
    try:
        import jwt
        from jwt import PyJWKSet
    except (
        Exception
    ) as exc:  # pragma: no cover - only when the auth extra isn't installed
        raise EdgeAuthError(
            "PyJWT[crypto] is required for edge auth ? install the 'auth' extra "
            "(pip install -e .[auth]) on a host with edge_auth_enabled."
        ) from exc

    try:
        kid = jwt.get_unverified_header(token).get("kid")
        if not kid:
            raise EdgeAuthError("token header has no kid")
        signing_key = PyJWKSet.from_dict(jwks)[kid]
        claims = jwt.decode(
            token,
            signing_key.key,
            algorithms=["RS256"],
            audience=aud,
            issuer=_issuer(team_domain),
            options={"require": ["exp", "iss", "aud"]},
        )
    except EdgeAuthError:
        raise
    except KeyError as exc:  # kid not in the JWKS
        raise EdgeAuthError(f"no JWKS key for kid {exc}") from exc
    except (
        Exception
    ) as exc:  # jwt.* errors: bad sig / expired / wrong aud|iss / malformed
        raise EdgeAuthError(f"Cf-Access token verification failed: {exc}") from exc

    user = claims.get("email") or claims.get("sub")
    if not user:
        raise EdgeAuthError("verified token carries no email/sub claim")
    return str(user)


def resolve_tenant(
    headers: Mapping[str, str],
    config: Any,
    *,
    jwks_fetcher: Optional[JwksFetcher] = None,
) -> Optional[str]:
    """Resolve the tenant ``user_id`` for a request, or ``None`` when edge auth is OFF.
```

```
    DEFAULT-OFF (``security.edge_auth_enabled=False``) ? ``None`` (the local single-user; unchanged).
    Enabled ? require + verify the Cf-Access header; raise ``EdgeAuthError`` (? 401/403) on any miss.
    ``jwks_fetcher`` is injectable for offline tests.
    """
    sec = getattr(config, "security", None)
    if not sec or not getattr(sec, "edge_auth_enabled", False):
        return None  # DEFAULT-OFF ? no tenant identity, byte-identical to today

    # Strip so a whitespace-only value counts as unset ? matching the boot-time exposure check in
    # config/startup.py (EXPOSED_EDGE_AUTH_INCOMPLETE); one semantics for "configured", both places.
    team_domain = (getattr(sec, "cf_team_domain", "") or "").strip()
    aud = (getattr(sec, "cf_access_aud", "") or "").strip()
    if not team_domain or not aud:
        raise EdgeAuthError(
            "edge_auth_enabled but security.cf_team_domain / cf_access_aud are not configured"
        )

    # Headers are case-insensitive over HTTP; Starlette's Headers handles that, but support a plain
    # dict too (tests) by trying both casings.
    token = headers.get(CF_ACCESS_HEADER) or headers.get(CF_ACCESS_HEADER.lower())
    if not token:
        raise EdgeAuthError(
            f"missing {CF_ACCESS_HEADER} header (request did not pass Cloudflare Access)"
        )

    jwks = _get_jwks(team_domain, jwks_fetcher)
    return verify_cf_access_token(token, team_domain=team_domain, aud=aud, jwks=jwks)

```

22. user (2026-07-02T02:49:38.217Z)

```
backend\main.py:731:    SELF-HOST posture only (D7): refused with 403 when ``security.allowed_video_root`` is set ? the
backend\main.py:747:    if getattr(sec, "allowed_video_root", None):
backend\main.py:752:                "(security.allowed_video_root is set); upload to local instead."
backend\main.py:1373:        getattr(request.app.state.config, "security", None), "allowed_video_root", None
backend\main.py:1388:    # input: its null-byte guard always applies, and a configured allowed_video_root (hosted mode)
backend\main.py:1797:    auth on without the ``allowed_video_root`` jail / CF team+AUD). A strict no-op when no
backend\config\models.py:330:    allowed_video_root: Optional[str] = None
backend\config\models.py:334:    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
backend\config\startup.py:22:rest of the posture at **boot**: the ``allowed_video_root`` path-jail is set (else an authenticated
backend\config\startup.py:107:    # a tenant can point the engine at an ARBITRARY local file (allowed_video_root unset = "any local
backend\config\startup.py:109:    root = getattr(sec, "allowed_video_root", None)
backend\config\startup.py:115:                "security.edge_auth_enabled is ON (engine exposed) but security.allowed_video_root is "
backend\config\startup.py:117:                "Set allowed_video_root to a jail directory before exposing the engine.",
backend\utils\paths.py:58:def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
```

23. user (2026-07-02T02:49:43.633Z)

```
"""Path-safety helpers for the API boundary (untrusted request fields)."""

import os
from typing import Any, Optional


def output_base(config: Any) -> str:
    """The configured base dir for a segmenter's LOCAL scratch/staging artifacts
    (COMPUTE_DECOUPLED_SERVING M3). Returns ``output.output_dir`` (default ``"output"``) so the
    chunk-staging / AI-handoff / temp dirs follow ``--output-dir`` / ``output.output_dir`` instead of
    a hardcoded ``./output``. The DEFAULT is ``"output"`` ? byte-identical to the prior literal.

    This is the LOCAL output dir, deliberately NOT the storage ``workspace_root`` (which may be a
    ``gs://`` URI you can't ``makedirs``). Accepts the typed AppConfig or the legacy dict-shim
    (``.get("output", {}).get("output_dir")``); any miss or empty value falls back to ``"output"``."""
    try:
        out = (
            config.output.output_dir
            if not isinstance(config, dict)
            else config.get("output", {}).get("output_dir")
        )
    except Exception:
        out = None
    return out or "output"


def safe_output_filename(name: str) -> str:
    """Validate an output filename is a BARE name (no dir part, no traversal, not absolute).

    `os.path.join(output_dir, name)` silently discards `output_dir` when `name` is absolute,
    and `..` segments escape it ? so an unchecked `output_filename` is an arbitrary-write
    primitive. Returns the name unchanged when safe; raises ValueError otherwise.
    """
    if not name or not name.strip():
        raise ValueError("output_filename must not be empty")
    if "\x00" in name:
        raise ValueError("output_filename must not contain a null byte")
    if "/" in name or "\\" in name:
        raise ValueError(f"output_filename must not contain a path separator: {name!r}")
    if os.path.isabs(name) or name != os.path.basename(name) or name in (".", ".."):
        raise ValueError(f"output_filename must be a bare filename, got: {name!r}")
    return name


def resolve_under_root(path: str, root: str) -> str:
    """Return the real path, asserting it is contained under `root` (raises ValueError if not)."""
    rp = os.path.realpath(path)
    rr = os.path.realpath(root)
    try:
        contained = os.path.commonpath([rp, rr]) == rr
    except ValueError:  # different drives on Windows
        contained = False
    if not contained:
```

```
            raise ValueError(f"path {path!r} escapes the allowed root {root!r}")
        return rp


    def validate_input_path(path: str, allowed_root: Optional[str] = None) -> None:
        """Reject null bytes and (when an allowed_root is configured) out-of-root inputs.

        With no allowed_root (default, single-user local mode) only the null-byte check applies,
        so the local 'point it at any of my videos' workflow is preserved. Setting an
        allowed_root (a future hosted/tenant deployment) enforces containment.
        """
        if "\x00" in path:
            raise ValueError("video_path must not contain a null byte")
        if allowed_root:
            resolve_under_root(path, allowed_root)

```

24. user (2026-07-02T02:49:44.530Z)

```
        client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
        runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
        worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
        """
        # M9 (D9): edge-auth tenant resolution. DEFAULT-OFF ? owner_id None (the local single-user,
        # unchanged). When edge_auth_enabled, the Cf-Access JWT is required + verified (a missing or
        # spoofed identity header fails here with 401, before any work) and tags the job's owner_id.
        try:
            owner_id = edge_auth.resolve_tenant(request.headers, request.app.state.config)
        except edge_auth.EdgeAuthError as e:
            raise HTTPException(status_code=401, detail=f"edge auth: {e}") from e

        allowed_root = getattr(
            getattr(request.app.state.config, "security", None), "allowed_video_root", None
        )
        try:
            validate_input_path(req.video_path, allowed_root=allowed_root)
            # M2: resolve the input once (path or storage URI) ? raises ValueError for an unsupported
            # scheme, which we map to a clean 400 (an unknown scheme must never 500).
            input_ref = storage.resolve_input_ref(
                req.video_path, getattr(request.app.state.config, "storage", None)
            )
        except ValueError as e:
            raise HTTPException(status_code=400, detail=str(e)) from e
        # The local-existence fast-fail applies only to a LOCAL input, and stats the RESOLVED local path
        # (input_ref.key ? e.g. local://? stripped to the bare path), not the raw URI string. A
        # bucket/Drive URI can't be cheaply os.path.exists()'d ? its existence is verified when the worker
        # fetches it (a fetch miss fails the job loudly). validate_input_path above still runs for every
        # input: its null-byte guard always applies, and a configured allowed_video_root (hosted mode)
        # rejects a non-local URI as out-of-root.
        if input_ref.backend == "local" and not os.path.exists(input_ref.key):
            raise HTTPException(
                status_code=404, detail=f"Video file not found at '{req.video_path}'"
            )
        # P2 (D11): a bucket/Drive source is fetched to a scratch copy on the worker; fail fast with 507
```

```
        # if this box can't hold that copy. Best-effort ? an unstattable remote (network/SDK) or an
        # unreadable temp fs skips the guard (measuring must not deny a legit job); the fetch itself
        # still fails loudly downstream if space genuinely runs out. Local inputs need no copy ? skipped.
        if input_ref.backend != "local":
            need = _remote_input_size_bytes(
                input_ref, getattr(request.app.state.config, "storage", None)
            )
            require_free_bytes(tempfile.gettempdir(), need, stage="fetch")
        if req.segmenter and not segmenter_registry.get(req.segmenter):
            available = list(segmenter_registry.list_available().keys())
            raise HTTPException(
                status_code=400,
                detail=f"Segmenter '{req.segmenter}' not found. Available segmenters: {available}",
            )

        job_id = uuid.uuid4().hex
        if not request.app.state.db.create_job(
            job_id,
            req.video_path,
            req.segmenter,
            req.player_pool,
            req.max_frames,
            owner_id=owner_id,
            locale=req.locale,
            compute=req.compute,
```

25. user (2026-07-02T02:49:55.154Z)

```
deploy\gcp\README.md:93:> **always bakes in the Ops Agent** (`--metadata-from-file startup-script=deploy/gcp/ops-agent-startup.sh`)
deploy\gcp\README.md:144:> `--metadata=startup-script` so observability is automatic on every box.
deploy\gcp\README.md:151:> driver: either add `--metadata=install-nvidia-driver=True` style startup, or after SSH run GCP's
deploy\gcp\README.md:187:mkdir -p /root/.keys && printf '%s' '<ROTATED_GEMINI_KEY>' > /root/.keys/GEMINI_API_KEY && chmod 600 /root/.keys/GEMINI_API_KEY
deploy\gcp\ops-agent-startup.sh:5:# safe. Baked into EVERY rally GPU VM by deploy/gcp/create_gpu_vm.sh (--metadata-from-file
deploy\gcp\create_gpu_vm.sh:3:# spin up a rally GPU box ? it bakes the ops-agent startup-script into the VM metadata, attaches the
deploy\gcp\create_gpu_vm.sh:49:       --metadata-from-file=startup-script="$STARTUP" 2>"$ERR"; then
deploy\digitalocean\README.md:29:| 6 | **`GEMINI_API_KEY`** | Google AI Studio (or Vertex) | the Gemini segmenter | **later** ? only to test the *Gemini* path; the stub/WASB path needs none. **Rotate the old key that was shared in chat.** |
deploy\digitalocean\bootstrap.sh:42:# `trackers` package. Git-only dep (PyPI wheel is metadata-only ? roboflow/trackers#474),
deploy\gcp\nightly_regression\launch.sh:60:  echo "[dry-run] staged $TGZ_URI OK; would create $GPU VM '$NAME' with run_on_vm.sh + metadata. Skipping VM create."
deploy\gcp\nightly_regression\launch.sh:64:# 2) Zone-sweep create the VM (mirrors create_gpu_vm.sh) with run_on_vm.sh + the config metadata.
deploy\gcp\nightly_regression\launch.sh:74:       --metadata-from-file=startup-script="$STARTUP" \
deploy\gcp\nightly_regression\launch.sh:75:       --metadata=repo-tgz-uri="$TGZ_URI",git-sha="$SHA",report-bucket="$NIGHTLY_BUCKET",report-prefix="$REPORT_PREFIX",weights-uri="$WEIGHTS_URI",email="$DO_EMAIL",smtp-secret="$SMTP_SECRET",smtp-user="$SMTP_USER",email-to="$EMAIL_TO" \
deploy\gcp\nightly_regression\README.md:140:| `reels=0`, log mentions GEMINI_API_KEY | `skip_ai_handoff` not true ? ensure the manifest `config_overrides` has `indexing.skip_ai_handoff: true` |
deploy\gcp\nightly_regression\run_on_vm.sh:4:# an interactive session). Reads its config from
```

```
instance metadata (set by launch.sh).
deploy\gcp\nightly_regression\run_on_vm.sh:21:
"http://metadata.google.internal/computeMetadata/v1/$1"; }
deploy\gcp\nightly_regression\run_on_vm.sh:70:if [ -z "$REPO_TGZ" ]; then echo "FATAL: repo-tgz-uri metadata missing ? nothing to run."; exit 1; fi
```

26. user (2026-07-02T02:49:56.741Z)

```
scripts\run_phase3_eval.py:14:Full corpus with the two-stage detector (needs GEMINI_API_KEY; costs API calls):
scripts\run_phase3_eval.py:31:load_dotenv()  # make GEMINI_API_KEY (etc.) available to the AI-backed segmenters
scripts\run_phase3_eval.py:65:        "yolo_hybrid/gemini need GEMINI_API_KEY and cost API calls.",
tools\generate_highlights.py:109:    # Load .env (e.g. GEMINI_API_KEY) so the Gemini handoff works when the runner is invoked
```

27. user (2026-07-02T02:50:06.426Z)

```
#!/usr/bin/env bash
# Launch ONE nightly engine-regression run on an ephemeral GCP GPU VM, then return. The VM runs
# run_on_vm.sh as its startup-script (regression ? email ? SELF-DELETE), so this just stages the repo
# and kicks the VM off. Driver-agnostic: call it from Cloud Scheduler?Cloud Run job, a cron on any
# always-on box, the owner's machine, or a Claude routine. Needs gcloud auth + the repo checkout.
#
#   bash deploy/gcp/nightly_regression/launch.sh                 # L4, stages HEAD, kicks the VM
#   RALLY_GPU=t4 RALLY_REF=$(git rev-parse HEAD) bash deploy/gcp/nightly_regression/launch.sh
#   RALLY_DRY_RUN=1 bash deploy/gcp/nightly_regression/launch.sh  # stage only, DON'T create the VM (smoke test)
#
# Works BOTH from a local git checkout (archives REF) AND from the Cloud Run launcher container, which has
# no .git (it tars the baked tree + takes the SHA from $RALLY_GIT_SHA ? set by register_scheduler.sh).
#
# ? The VM-side flow is NOT covered by CI. Tail: gcloud compute instances get-serial-port-output <name> --zone=<z>.
set -uo pipefail

# Centralized project + bucket names (single source of truth ? deploy/gcp/buckets.env). Now honors
# RALLY_CORPUS_BUCKET (canonical) as well as the legacy RALLY_BUCKET / RALLY_GCS_BUCKET.
source "$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)/../buckets.env"

PROJECT="$RALLY_GCP_PROJECT"
CORPUS_BUCKET="$RALLY_CORPUS_BUCKET"    # PERMANENT inputs: clips/labels/weights (no TTL)
NIGHTLY_BUCKET="$RALLY_NIGHTLY_BUCKET"  # EPHEMERAL outputs: reports + repo tarball (30d TTL)
SA="rally-worker@${PROJECT}.iam.gserviceaccount.com"
GPU="${RALLY_GPU:-l4}"
NAME="${RALLY_VM_NAME:-rally-nightly-$(date -u +%Y%m%d-%H%M%S)}"   # to-the-second ? same-day re-runs don't collide
REF="${RALLY_REF:-HEAD}"
WEIGHTS_URI="$RALLY_WEIGHTS_URI"  # gs://<corpus>/models/wasb_badminton_best.pth.tar (derived in buckets.env; validated 2026-06-22)
REPORT_PREFIX="${RALLY_REPORT_PREFIX:-reports}"
DO_EMAIL="${RALLY_EMAIL:-true}"
SMTP_SECRET="${NIGHTLY_SMTP_SECRET:-projects/$PROJECT/secrets/nightly-smtp-
```

```
pass/versions/latest}"
SMTP_USER="${NIGHTLY_SMTP_USER:-}"
EMAIL_TO="${NIGHTLY_EMAIL_TO:-avi.dullu@gmail.com}"
DRY_RUN="${RALLY_DRY_RUN:-}"
HERE="$(cd "$(dirname "${BASH_SOURCE[0]}")" && pwd)"
ROOT="$(cd "$HERE/../../.." && pwd)"   # repo root (deploy/gcp/nightly_regression ? 3 up)
STARTUP="$HERE/run_on_vm.sh"
export CLOUDSDK_CORE_DISABLE_PROMPTS=1

ZONES="${RALLY_GCP_ZONES:-asia-south1-a asia-south1-b asia-south1-c asia-southeast1-a asia-southeast1-b asia-southeast1-c}"
if [ "$GPU" = "l4" ]; then MACHINE="g2-standard-4"; ACCEL=(); else MACHINE="n1-standard-8"; ACCEL=(--accelerator=type=nvidia-tesla-t4,count=1); fi

# 1) Stage the repo to the EPHEMERAL nightly bucket (30d TTL; no GitHub auth on the VM).
#    Create the bucket first if missing: bash deploy/gcp/nightly_regression/create_nightly_bucket.sh
#    Two modes: a git checkout archives the tracked tree @ REF; the Cloud Run container (no .git) tars
#    the baked tree and takes the SHA from $RALLY_GIT_SHA / a baked GIT_SHA file.
SHA="${RALLY_GIT_SHA:-$( (cd "$ROOT" && git rev-parse "$REF" 2>/dev/null) || cat "$ROOT/GIT_SHA" 2>/dev/null || echo unknown )}"
TGZ_URI="gs://$NIGHTLY_BUCKET/repo/${SHA}.tgz"
echo "== staging repo $SHA -> $TGZ_URI =="
TMP_TGZ="$(mktemp --suffix=.tgz)"
if (cd "$ROOT" && git rev-parse --git-dir >/dev/null 2>&1); then
  ( cd "$ROOT" && git -c core.autocrlf=false archive --format=tar.gz -o "$TMP_TGZ" "$REF" )   # local checkout
else
  tar czf "$TMP_TGZ" -C "$ROOT" --exclude='./.git' .                       # container: baked tree
fi
gcloud storage cp "$TMP_TGZ" "$TGZ_URI" --project="$PROJECT"
rm -f "$TMP_TGZ"

if [ -n "$DRY_RUN" ]; then
  echo "[dry-run] staged $TGZ_URI OK; would create $GPU VM '$NAME' with run_on_vm.sh + metadata. Skipping VM create."
  exit 0
fi

# 2) Zone-sweep create the VM (mirrors create_gpu_vm.sh) with run_on_vm.sh + the config metadata.
ERR="$(mktemp)"
for Z in $ZONES; do
  echo "== creating $NAME ($GPU) in $Z =="
  if gcloud compute instances create "$NAME" --project="$PROJECT" --zone="$Z" \
       --machine-type="$MACHINE" "${ACCEL[@]}" \
       --maintenance-policy=TERMINATE --provisioning-model=STANDARD \
       --image-family=common-cu129-ubuntu-2204-nvidia-580 --image-project=deeplearning-platform-release \
       --boot-disk-size=120GB --boot-disk-type=pd-balanced \
       --service-account="$SA" --scopes=cloud-platform \
       --metadata-from-file=startup-script="$STARTUP" \
       --metadata=repo-tgz-uri="$TGZ_URI",git-sha="$SHA",report-bucket="$NIGHTLY_BUCKET",report-prefix="$REPORT_PREFIX",weights-uri="$WEIGHTS_URI",email="$DO_EMAIL",smtp-secret="$SMTP_SECRET",smtp-user="$SMTP_USER",email-to="$EMAIL_TO" \
       2>"$ERR"; then
    echo "? launched $NAME in $Z ? it will run the regression, email $EMAIL_TO, and self-delete."
    echo "   serial log: gcloud compute instances get-serial-port-output $NAME --zone=$Z --project=$PROJECT"
    echo "   if it doesn't self-delete: gcloud compute instances delete $NAME --zone=$Z
```

```
-q"
          rm -f "$ERR"; exit 0
        fi
        if grep -qiE "STOCKOUT|does not have enough" "$ERR"; then echo "  (no $GPU capacity in
$Z ? next zone)"; continue; fi
        echo "  create failed:"; tail -4 "$ERR"; rm -f "$ERR"; exit 1
      done
      echo "? no $GPU capacity in any swept zone ($ZONES). Retry later or RALLY_GPU=t4."
      rm -f "$ERR"; exit 1
```

28. user (2026-07-02T02:50:48.674Z)

```
41:from fastapi.staticfiles import StaticFiles
240:app.mount("/static", StaticFiles(directory=_STATIC_DIR), name="static")
255:    app.mount("/assets", StaticFiles(directory=_UI_ASSETS_DIR), name="assets")
```

29. user (2026-07-02T02:50:50.170Z)

```
No matches found
```

30. user (2026-07-02T02:50:51.853Z)

```
8615534 refactor: adopt approved layered directory structure
72f972e 2026-05-18 Checkpoint: Add debug trim feature, runner script, and motion analysis
scratch script
```

Coverage data

```
.coverage
.coverage.*
htmlcov/
.pytest_cache/
```

Local Claude Code config (machine-specific)

```
.claude/
```

Throwaway repro scripts + PR-body / handoff scratch notes (already excluded from ruff + pytest; never meant to be committed -- keep them out of `git status` too).

```
scratch/
.venv/
```

Private leak-scan name-list (real player/venue names for LOCAL `tools.leak_scan --nar scanning -- NEVER committed; CI runs the name-free structural guard).

31. user (2026-07-02T02:51:18.624Z)

```
CLAUDE.md
README.md
__pycache__
artifacts
backend
backend.db
config.json
deploy
docs
eval_baselines
indexer.db
mypy.ini
output
pyproject.toml
pytest.ini
```

```
rally_index.db
requirements.txt
ruff.toml
run_all_tests.py
run_eval.py
run_regression.py
scratch
scripts
tests
tools
training
requirements.txt
deploy/gcp/create_gpu_vm.sh:20:# Centralized project + bucket names (single source of truth ?
deploy/gcp/buckets.env).
deploy/gcp/create_gpu_vm.sh:21:source "$(cd "$(dirname "${BASH_SOURCE[0]}")" &&
pwd)/buckets.env"
deploy/gcp/nightly_regression/create_nightly_bucket.sh:16:# Centralized project + bucket names
(single source of truth ? deploy/gcp/buckets.env).
deploy/gcp/nightly_regression/create_nightly_bucket.sh:17:source "$(cd "$(dirname
"${BASH_SOURCE[0]}")" && pwd)/../buckets.env"
deploy/gcp/nightly_regression/launch.sh:17:# Centralized project + bucket names (single source
of truth ? deploy/gcp/buckets.env). Now honors
deploy/gcp/nightly_regression/launch.sh:19:source "$(cd "$(dirname "${BASH_SOURCE[0]}")" &&
pwd)/../buckets.env"
deploy/gcp/nightly_regression/README.md:54:| **Buckets** | corpus (inputs) + `khelsutra-rally-
nightly` (outputs) | `RALLY_CORPUS_BUCKET` / `RALLY_NIGHTLY_BUCKET` ? the single source of truth
is `deploy/gcp/buckets.env` (sourced by every script). `RALLY_BUCKET` + `RALLY_GCS_BUCKET`
remain accepted back-compat aliases for the corpus bucket |
deploy/gcp/nightly_regression/register_scheduler.sh:17:# Centralized project + bucket names
(single source of truth ? deploy/gcp/buckets.env).
deploy/gcp/nightly_regression/register_scheduler.sh:18:source "$(cd "$(dirname
"${BASH_SOURCE[0]}")" && pwd)/../buckets.env"
deploy/gcp/nightly_regression/setup_email.sh:16:# Centralized project + bucket names (single
source of truth ? deploy/gcp/buckets.env).
---
deploy/gcp/provision.sh:14:PROJECT="$RALLY_GCP_PROJECT"
deploy/gcp/create_gpu_vm.sh:23:PROJECT="$RALLY_GCP_PROJECT"
deploy/gcp/teardown.sh:25:PROJECT="$RALLY_GCP_PROJECT"
deploy/gcp/nightly_regression/register_scheduler.sh:20:PROJECT="$RALLY_GCP_PROJECT"
deploy/gcp/nightly_regression/create_nightly_bucket.sh:19:PROJECT="$RALLY_GCP_PROJECT"
```

32. user (2026-07-02T02:51:36.032Z)

```
---ignore-check---
.gitignore:44:*.db      backend.db
.gitignore:44:*.db      indexer.db
.gitignore:44:*.db      rally_index.db
.gitignore:47:output/   output
.gitignore:2:__pycache__/       __pycache__
---gemini keyfile refs---
187:mkdir -p /root/.keys && printf '%s' '<ROTATED_GEMINI_KEY>' > /root/.keys/GEMINI_API_KEY &&
chmod 600 /root/.keys/GEMINI_API_KEY
```

33. user (2026-07-02T02:51:44.606Z)

```
deploy\gcp\nightly_regression\register_scheduler.sh:50:for ROLE in
roles/compute.instanceAdmin.v1 roles/iam.serviceAccountUser roles/run.invoker; do
```

34. user (2026-07-02T02:51:44.914Z)

```
1       # Cloud Build config for the Cloud Run GPU worker image (COMPUTE_DECOUPLED_SERVING M7).
2       #
3       # `gcloud builds submit --tag ? -f ?` does NOT work ? `gcloud builds submit` has no `-f`
flag, so a
```

```yaml
# non-root Dockerfile needs a build CONFIG like this one. Build with:
#
#   gcloud builds submit --config=deploy/cloudrun/cloudbuild.yaml --project <PROJECT> .
#
# Override the region/repo/image with --substitutions=_REGION=?,_REPO=?,_IMAGE=? if needed.
# PROJECT_ID is provided by Cloud Build automatically.
substitutions:
  _REGION: asia-southeast1   # L4 GPU is region-limited (asia-southeast1 / europe-west1 / europe-west4 / us-central1 / us-east4)
  _REPO: rally
  _IMAGE: rally-worker
steps:
  - name: 'gcr.io/cloud-builders/docker'
    args:
      - build
      - -t
      - '${_REGION}-docker.pkg.dev/${PROJECT_ID}/${_REPO}/${_IMAGE}:latest'
      - -f
      - deploy/cloudrun/Dockerfile
      - .
images:
  - '${_REGION}-docker.pkg.dev/${PROJECT_ID}/${_REPO}/${_IMAGE}:latest'
options:
  machineType: E2_HIGHCPU_8   # the CUDA + micromamba image is heavy; a bigger builder keeps it ~8?10 min
timeout: 3600s

```

35. user (2026-07-02T02:53:37.550Z)

Structured output provided successfully